Tool System

Claude Code Agent and Tool System Design Research Report

Table of Contents

1. Overall Architecture
    - 1.1 Top-level Directory Structure
    - 1.2 Request Entry Flow
    - 1.3 Data Flow Overview
2. Tool System
    - 2.1 Core Type: `Tool<Input, Output>`
    - 2.2 `buildTool` Factory Function
    - 2.3 Tool Pool Assembly (`tools.ts`)
    - 2.4 Tool Execution Pipeline (`services/tools/`)
    - 2.5 StreamingToolExecutor Concurrency Model
    - 2.6 runToolUse Execution Details
    - 2.7 All 44 Tools Overview
    - 2.8 Tool Constants and Access Control
    - 2.9 Tool System Design Philosophy Analysis
    - 2.10 Tool Execution Sequence Diagram (Supplementary)
3. Agent System
    - 3.1 AgentTool Overview
    - 3.2 Supported Agent Types and Spawn Modes
    - 3.3 Input/Output Schema
    - 3.4 Sync vs. Async Agent Differences
    - 3.5 Worktree Isolation
    - 3.6 Agent Definition File Format
    - 3.7 Built-in Agents
    - 3.8 runAgent Execution Engine
    - 3.9 Fork Subagent Feature
    - 3.10 Agent Memory System
    - 3.11 AgentTool Constants
    - 3.12 Agent System Design Philosophy Analysis
4. Multi-Agent Collaboration and Teams
    - 4.1 Team Tools
    - 4.2 Swarm Backend System (`utils/swarm/`)
    - 4.3 Teammate Spawn Flow (`tools/shared/spawnMultiAgent.ts`)
    - 4.4 SendMessageTool Communication Protocol
    - 4.5 Shutdown Protocol
    - 4.6 AgentContext and AsyncLocalStorage
5. Task Framework
    - 5.1 Task Type System
    - 5.2 LocalAgentTask Complete State Machine
    - 5.3 Core Operations
    - 5.4 Progress Tracking
    - 5.5 Message Queue and Delivery
    - 5.6 Lifecycle Hooks
6. Coordinator Mode
    - 6.1 Overview
    - 6.2 Complete System Prompt Structure
    - 6.3 Spawn vs. Continue Decision Principles
    - 6.4 Tool Constraints Summary
7. Query Engine
    - 7.1 QueryEngine Class
    - 7.2 submitMessage Execution Flow
    - 7.3 query.ts Main Query Loop
    - 7.4 queryLoop State Machine
    - 7.5 Turn Loop (while(true))
    - 7.6 Key Recovery Mechanisms
    - 7.7 Query Engine Design Philosophy Analysis
8. MCP Integration (Model Context Protocol)
    - 8.1 MCP Client Architecture (`services/mcp/`)
    - 8.2 MCP OAuth Authentication Flow
    - 8.3 MCP Resource Tools
9. Skill System
    - 9.1 Skill Definition
    - 9.2 Skill Loading
    - 9.3 SkillTool
    - 9.4 Built-in Bundled Skills
    - 9.5 Dynamic Skill Discovery
10. Plugin System
    - 10.1 Plugin Manifest Schema
    - 10.2 Loaded Plugin Types
    - 10.3 Plugin Source Types
    - 10.4 Plugin Loading Pipeline
    - 10.5 Plugin Error Types
11. Permission and Security Model
    - 11.1 Permission Modes
    - 11.2 Permission Behaviors
    - 11.3 Permission Decision Types
    - 11.4 Permission Decision Reasons
    - 11.5 Auto Classifier
    - 11.6 Permission Context
    - 11.7 Hook Permission Decision Resolution
    - 11.8 Bash Tool Security Analysis
12. Core Type Definitions
    - 12.1 Tool-related Core Types
    - 12.2 Agent-related Core Types
    - 12.3 Task-related Core Types
    - 12.4 SDK Types (`entrypoints/agentSdkTypes.ts`)
13. Design Philosophy Summary
    - 13.1 Tool Design Philosophy
    - 13.2 Agent Design Philosophy
    - 13.3 Multi-Agent Collaboration Philosophy
    - 13.4 Query Engine Design
    - 13.5 MCP Ecosystem Design
    - 13.6 Plugin Architecture
    - 13.7 Hook System
    - 13.8 Global Design Philosophy of the Tool System
    - 13.9 Global Design Philosophy of the Agent System

- 13.10 Architectural Insights on Multi-Agent Collaboration
- 13.11 Fault Tolerance Philosophy of the Query Engine
- 13.12 Task Framework: State Machine as Lifecycle
- 13.13 Permission System: Layered Decision Tree
- 13.14 Memory System: Three Paradigms for Persistent Context

14. In-Depth Topic Analysis
    - 14.1 Six Design Principles of the Tool System
    - 14.2 Six Design Principles of the Agent System
    - 14.3 "Magic Comments" in Source Code: Reflection of Engineering Culture
    - 14.4 Quick Reference: Design Intent of Key Files
    - 14.5 Architecture Decision Records (ADR)

---

1. Overall Architecture

1.1 Top-level Directory Structure

Directory	Responsibilities
`tools/`	Definitions and registration of 44 tools
`commands/`	88 CLI command implementations (`/commit`, `/verify`, `/bug`, etc.)
`services/`	Core services: API calls, MCP client/server, Analytics, Compact/Memory
`utils/`	Utility functions: Git, Shell, Swarm (multi-Agent backend), Hooks, Settings
`tasks/`	Task framework: LocalAgentTask, RemoteAgentTask, InProcessTeammateTask
`coordinator/`	Multi-Agent Coordinator mode
`cli/`	CLI handler, transport layer, REPL
`components/`	React UI components (messages, permissions, Agent display)
`context/`	React Context
`plugins/`	Plugin system
`skills/`	Skill system
`assistant/`	KAIROS assistant mode
`buddy/`	AI companion UI
`remote/`	Remote session management
`voice/`	Voice interaction
`entrypoints/`	SDK entry points: `agentSdkTypes.ts`, `mcp.ts`

1.2 Request Entry Flow

```
main.tsx (Commander.js CLI parsing)
  └─> replLauncher.tsx
        └─> QueryEngine (main engine class)
              └─> submitMessage() [AsyncGenerator<SDKMessage>]
                    └─> query() from query.ts (main query loop)
                          ├─> callModel() [API streaming call]
                          ├─> StreamingToolExecutor [concurrent tool execution]
                          │     └─> runToolUse() [permissions/hooks/error handling]
                          │           └─> Tool.call() [each tool implementation]
                          └─> [loop back until completion or interruption]
```

1.3 Data Flow Overview

```
User input
  └─> submitMessage() [QueryEngine]
        ├─> processUserInput [process slash commands]
        ├─> recordSidechainTranscript() [persistence]
        └─> query() [AsyncGenerator]
              ├─> API streaming response (message_start/delta/stop events)
              ├─> executePostSamplingHooks()
              ├─> StreamingToolExecutor [concurrent tool execution]
              ├─> Context compression [HISTORY_SNIP / CONTEXT_COLLAPSE]
              ├─> AutoCompact [automatic compression]
              └─> Token Budget [continue or terminate decision]
                    └─> SDKMessage yielded one by one
                          └─> CLI UI rendering / SDK return
```

---

2. Tool System

2.1 Core Type: `Tool<Input, Output>`

All tools implement the `Tool` interface (defined in `Tool.ts`), which is the cornerstone of the entire system. The interface contains approximately **40 properties and methods**, divided into the following dimensions:

Identity and Schema

```
export type Tool<
  Input extends AnyObject = AnyObject,
  Output = unknown,
  P extends ToolProgressData = ToolProgressData,
> = {
  // === Identity ===
  name: string                                          // Unique tool name
  aliases?: string[]                                    // Backward-compatible rename aliases
  searchHint?: string                                   // ToolSearch keyword matching hint

  // === Schema ===
  readonly inputSchema: Input                           // Zod schema, defines tool input params
  readonly inputJSONSchema?: ToolInputJSONSchema        // JSON Schema representation
  outputSchema?: z.ZodType<unknown>                    // Zod schema, defines tool output

  // === Execution ===
  call(
    args: z.infer<Input>,
    context: ToolUseContext,
    canUseTool: CanUseToolFn,
    parentMessage: AssistantMessage,
    onProgress?: (progress: ToolProgress<P>) => void,
  ): Promise<ToolResult<Output>>

  // === Description Generation (LLM auto-understands tool) ===
```

```
  description(input: z.infer<Input>, options: ToolUseContext): Promise<string>
  prompt(options: ToolUseContext): Promise<string>       // Additional prompt
```

Concurrency and Safety

```
  // === Concurrency Safety ===
  isConcurrencySafe(input): boolean       // Whether it can run concurrently with other tools
  isReadOnly(input): boolean              // Whether read-only (no filesystem/state modification)
  isDestructive?(input): boolean          // Whether it's a destructive operation
  interruptBehavior?(): 'cancel' | 'block' // On interruption: cancel or block

  // === Enable Control ===
  isEnabled(): boolean                    // Feature gate
  inputsEquivalent?(a, b): boolean       // Input equivalence (deduplication)
```

Permissions and Paths

```
  // === Permissions ===
  checkPermissions(
    input: z.infer<Input>,
    context: ToolUseContext,
  ): Promise<PermissionResult>            // Core permission check
  validateInput?(input, context): Promise<ValidationResult>  // Custom validation
  getPath?(input): string                 // Extract operation path (for permission matching)
  preparePermissionMatcher?(input): Promise<(pattern: string) => boolean>
  backfillObservableInput?(input): void  // Fill missing compatibility fields
```

User-facing Information

```
  // === User-facing ===
  userFacingName(input): string                     // Display name
  userFacingNameBackgroundColor?(input): keyof Theme
  getToolUseSummary?(input): string | null          // Operation summary
  getActivityDescription?(input): string | null     // Activity description
  toAutoClassifierInput(input): unknown             // Auto-classifier features
```

Message Rendering (React UI)

```
  // === React Rendering ===
  isTransparentWrapper?(): boolean                   // Whether transparent wrapper (hide call)
  mapToolResultToToolResultBlockParam(content, toolUseID): ToolResultBlockParam
  renderToolResultMessage?(content, progressMessages, options): React.ReactNode
  renderToolUseMessage?(input, options): React.ReactNode
  renderToolUseProgressMessage?(progressMessages, options): React.ReactNode
  renderToolUseQueuedMessage?(): React.ReactNode
  renderToolUseRejectedMessage?(input, options): React.ReactNode
  renderToolUseErrorMessage?(result, options): React.ReactNode
  renderGroupedToolUse?(toolUses, options): React.ReactNode | null
  renderToolUseTag?(input): React.ReactNode
  extractSearchText?(output): string
  isResultTruncated?(output): boolean
```

Search and MCP

```
  // === Search/Read Classification ===
  isSearchOrReadCommand?(input): {
    isSearch: boolean; isRead: boolean; isList?: boolean
  }

  // === MCP Metadata ===
  readonly mcpInfo?: { serverName: string; toolName: string }
  readonly isMcp?: boolean
  readonly isLsp?: boolean
  readonly strict?: boolean        // Strict mode (tengu_tool_pear)

  // === Loading Strategy ===
  readonly shouldDefer?: boolean    // Deferred loading, triggered by ToolSearchTool
  readonly alwaysLoad?: boolean     // Never deferred
  readonly maxResultSizeChars: number
```

2.2 `buildTool` Factory Function

All tools are created via the `buildTool(def)` factory, which auto-fills safe defaults:

```
const TOOL_DEFAULTS = {
  isEnabled:           () => true,           // Enabled by default
  isConcurrencySafe:   () => false,          // Not concurrent by default (safe)
  isReadOnly:          () => false,          // Has write operations by default
  isDestructive:       () => false,
  checkPermissions:    (input, _ctx) => ({ behavior: 'allow', updatedInput: input }),
  toAutoClassifierInput: (_input) => '',     // Skip classifier by default
  userFacingName:      (_input) => name,     // Default to tool name
}

export function buildTool<D extends AnyToolDef>(def: D): BuiltTool<D> {
  return {
    ...TOOL_DEFAULTS,
    userFacingName: () => def.name,           // Forced override
    ...def,
  } as BuiltTool<D>
}
```

Design Principle: Fail-closed by default (conservative), each tool overrides as needed.

2.3 Tool Pool Assembly (`tools.ts`)

```
export function getAllBaseTools(): Tools {
  // 1. Direct imports (unconditionally available)
  return [
    AgentTool,
    BashTool,
    FileReadTool,
    FileEditTool,
    FileWriteTool,
    // ...
    // 2. Feature-gated imports (environment variables or feature flags)
    ...(process.env.USER_TYPE === 'ant' ? [ConfigTool, TungstenTool] : []),
    ...(isEnvTruthy(process.env.ENABLE_LSP_TOOL) ? [LSPTool] : []),
    ...(isWorktreeModeEnabled() ? [EnterWorktreeTool, ExitWorktreeTool] : []),
    ...(isAgentSwarmsEnabled() ? [getTeamCreateTool(), getTeamDeleteTool()] : []),
    ...(process.env.NODE_ENV === 'test' ? [TestingPermissionTool] : []),
    // 3. MCP-related (dynamically injected)
    ListMcpResourcesTool,
    ReadMcpResourceTool,
    ...(isToolSearchEnabledOptimistic() ? [ToolSearchTool] : []),
  ]
}

export function assembleToolPool(permissionContext, mcpTools): Tools {
  const builtInTools = getTools(permissionContext)
  const allowedMcpTools = filterToolsByDenyRules(mcpTools, permissionContext)
  // Sort by name to ensure prompt cache stability
  // uniqBy ensures: built-in tools take priority (same name overrides MCP tools)
  return uniqBy(
    [...builtInTools].sort(byName).concat(allowedMcpTools.sort(byName)),
    'name',
  )
}
```

2.4 Tool Execution Pipeline (`services/tools/`)

```
StreamingToolExecutor [concurrency control]
  ├─ Maintains queued/executing/completed/yielded state
  ├─ canExecuteTool() — concurrency safety check
  │   Non-concurrency-safe tools: must run alone
  │   Concurrency-safe tools: can run in parallel
  └─ processQueue() — sequential scheduling

  runToolUse() [toolExecution.ts]
    ├─ findToolByName() — tool lookup
    ├─ inputSchema.safeParse() — Zod input validation
    ├─ tool.validateInput?() — custom validation
    ├─ backfillObservableInput?() — fill missing compatibility fields
    ├─ runPreToolUseHooks() — pre-hook
    ├─ resolveHookPermissionDecision() — permission decision
    ├─ canUseTool() — permission check
    ├─ tool.call() — execute business logic
    ├─ runPostToolUseHooks() — post-hook
    └─ Error handling: McpAuthError / AbortError / various exceptions
```

2.5 StreamingToolExecutor Concurrency Model

```
type TrackedTool = {
  id: string
  block: ToolUseBlock
  status: 'queued' | 'executing' | 'completed' | 'yielded'
  isConcurrencySafe: boolean
  promise?: Promise<void>
  results?: Message[]
  pendingProgress: Message[]
  contextModifiers?: Array<(context: ToolUseContext) => ToolUseContext>
}

// Concurrency decision logic
private canExecuteTool(isConcurrencySafe: boolean): boolean {
  const executing = this.tools.filter(t => t.status === 'executing')
  return (
    executing.length === 0 ||
    (isConcurrencySafe && executing.every(t => t.isConcurrencySafe))
  )
}
```

Rule interpretation:

- The first tool always executes immediately
- Subsequent concurrency-safe tools can start in parallel (max concurrency depends on the `executing.length === 0` window)
- Non-concurrency-safe tools (e.g., Bash write operations) must wait for all tools to complete
- Results are buffered in "tool enqueue order" then yielded (ensures LLM sees consistent output order)
- When a Bash tool errors, all concurrent tools are cancelled via `siblingAbortController.abort('sibling_error')`

2.6 runToolUse Execution Details

```
async function* runToolUse(toolUse, assistantMessage, canUseTool, toolUseContext) {

  // Phase 1: Input parsing
  const parsedInput = tool.inputSchema.safeParse(input)

  // Phase 2: Custom validation (can return error to reject)
  const isValidCall = await tool.validateInput?.(parsedInput.data, toolUseContext)
  if (isValidCall?.result === false) { /* return error */ }

  // Phase 3: Speculative execution (parallel with Phase 2/4)
  if (tool.name === BASH_TOOL_NAME) {
    startSpeculativeClassifierCheck(command) // Start classifier check early
  }

  // Phase 4: Pre-tool hooks (PreToolUse Hooks)
  for await (const result of runPreToolUseHooks(...)) {
    switch (result.type) {
```

```
      case 'message':         yield result.message        // Forward message
      case 'hookPermissionResult': permissionResult = result
      case 'hookUpdatedInput':    processedInput = result.updatedInput
      case 'preventContinuation': shouldStop = true
      case 'additionalContext':   yield additionalContext
      case 'stop':                return                   // Exit immediately
    }
  }

  // Phase 5: Permission decision
  const resolved = await resolveHookPermissionDecision(
    hookPermissionResult, tool, processedInput, toolUseContext, canUseTool, ...
  )
  if (resolved.decision.behavior !== 'allow') {
    return yield* handlePermissionDenied(...)  // Return rejection message
  }

  // Phase 6: Execution
  const result = await tool.call(callInput, toolUseContext, canUseTool, ...)

  // Phase 7: Post-tool hooks (PostToolUse Hooks)
  for await (const hookResult of runPostToolUseHooks(...)) {
    if ('updatedMCPToolOutput' in hookResult) {
      // MCP tool: update output
    } else {
      // Built-in tool: push message
    }
  }

  // Phase 8: Error handling
} catch (error) {
  // McpAuthError → update client state to 'needs-auth'
  // AbortError → mark interruption
  // Other errors → log and return error message
}
```

2.7 All 44 Tools Overview

Tool Name	Type	Description
AgentTool	Agent	Launch sub-agents (general, Fork, teammate, remote)
BashTool	Command	Shell commands, with sandbox, AST security analysis, concurrency classification
FileEditTool	Edit	File editing based on sed or patch, with TOCTOU protection
FileReadTool	Read	Restricted file reading
FileWriteTool	Write	File writing
GlobTool	Search	Pattern-matching file search
GrepTool	Search	File content text search
NotebookEditTool	Edit	Jupyter notebook editing
LSPTool	LSP	Language Server Protocol integration
MCPTool	MCP	Model Context Protocol tool wrapper
McpAuthTool	MCP	MCP OAuth authentication (dynamically created)
ListMcpResourcesTool	MCP	List MCP server resources
ReadMcpResourceTool	MCP	Read MCP resource content
WebFetchTool	Web	Fetch web page content
WebSearchTool	Web	Web search
TaskCreateTool	Task	Create task
TaskGetTool	Task	Get task details
TaskUpdateTool	Task	Update task status
TaskListTool	Task	List tasks
TaskOutputTool	Task	Read task output
TaskStopTool	Task	Stop running task
TeamCreateTool	Team	Create Agent team
TeamDeleteTool	Team	Delete team
SendMessageTool	Team	Send message to teammate (peer-to-peer/broadcast/structured protocol)
AskUserQuestionTool	Interaction	Request user input
ConfigTool	Config	Configuration management (ant exclusive)
TungstenTool	Config	Tungsten configuration (ant exclusive)
BriefTool	Task	Task briefing/tool
SkillTool	Skill	Invoke Skill system
ExitPlanModeTool	Mode	Exit plan mode
EnterPlanModeTool	Mode	Enter plan mode
EnterWorktreeTool	Git	Enter Git Worktree
ExitWorktreeTool	Git	Exit Git Worktree
TodoWriteTool	Todo	Write Todo list
ScheduleCronTool	Scheduling	Cron scheduling (Create/Delete/List)
RemoteTriggerTool	Remote	Remote Agent trigger
SyntheticOutputTool	Internal	Internal synthetic output
ToolSearchTool	Search	Tool search/deferred loading
PowerShellTool	Command	PowerShell commands
REPLTool	Interaction	REPL mode wrapper
WorkflowTool	Script	Workflow scripts
SleepTool	Auxiliary	Sleep/delay
TestingPermissionTool	Test	Test permission tool (test environment only)

2.9 Tool System Design Philosophy Analysis

2.9.1 The "Anti-pattern" Beauty of the Tool Interface: Maximum Cohesion

Note

Why pile all properties into one interface?

Core reason: **So the LLM can "see" the complete behavioral description of a tool in one shot.**

When the model needs to decide whether to call a tool, it relies on the tool's schema and prompts. If a tool's behavior is scattered across multiple mixins or inheritance hierarchies, the LLM has a hard time accurately inferring the full capability boundary of the tool (e.g., the combination of `isDestructive` and `interruptBehavior`). Claude Code chooses to concentrate all "how the tool sees itself" properties in the same interface, so each tool's self-description is **complete and cohesive**.

Costs and Benefits of This Design:

Cost	Benefit
Large interface (40+ properties)	LLM doesn't need to understand object inheritance
Some tools have many irrelevant properties	All tool-checking logic is unified, no polymorphic dispatch needed
Requires `buildTool` factory to fill defaults	New tool properties have controllable impact scope (only need to change the factory)

The default value strategy of the **`buildTool` factory** is the "safety foundation" of the entire tool system:

```
const TOOL_DEFAULTS = {
  isEnabled:           () => true,       // Enabled by default (not disabled by default)
  isConcurrencySafe:   () => false,      // Not concurrent by default (conservative)
  isReadOnly:          () => false,      // Has side effects by default (security-oriented)
  checkPermissions:    (input, _ctx) => ({ behavior: 'allow', updatedInput: input }),
  // ...
}
```

Key insight: Default `isEnabled: true` rather than false — this may seem to contradict the fail-closed principle, but actually "enabled" is the default state, and "disabled" requires explicit declaration. For infrequently used tools, Claude Code uses deferred loading (`shouldDefer`) rather than default disabling, which is a more elegant solution.

2.9.2 StreamingToolExecutor Concurrency Model: Minimum Safety Window

Note

StreamingToolExecutor's concurrency design is **not** a "maximum parallelism" model, but a "**minimum safety window**" model

Pseudocode logic for concurrency decision (simplified):

```
// First tool always executes immediately
// Second tool: only parallel if "first tool is concurrency-safe"
// Third and beyond: inherit the same constraints

canExecute(isConcurrencySafe) {
  const executing = this.tools.filter(t => t.status === 'executing')
  return executing.length === 0  // Zero executing → can start
      || (isConcurrencySafe && executing.every(t => t.isConcurrencySafe))
      // None of the above → wait (for non-concurrency-safe tools, this is all the conditions)
}
```

What does this mean?

- If the model calls 5 tools simultaneously, if any one is "non-concurrency-safe", the entire queue **serializes** at that tool
- Search tools (`isConcurrencySafe: true`) can bunch up in parallel
- Bash write operations (`isConcurrencySafe: false`) block all subsequent tools

The elegant design of `siblingAbortController`:

```
// When a Bash tool errors:
if (tool.block.name === BASH_TOOL_NAME) {
  this.hasErrored = true
  this.siblingAbortController.abort('sibling_error')
}
```

This `AbortController` is a **child** of `toolUseContext.abortController`, not a parent. When triggered:

- It terminates **other concurrent Bash child processes** (they listen to this signal)
- It does **NOT** terminate the parent query loop (`query.ts` is unaffected)
- It does NOT trigger ESC interruption (ESC interrupts the entire turn)

This is a **three-level interruption system**:

```
User ESC ──→ abortController (terminates entire turn)
  Bash error ──→ siblingAbortController (only terminates sibling bash processes)
    Permission denial ──→ toolAbortController (reports to query loop)
```

Note

Why only Bash tools can trigger sibling abort? By design, `siblingAbortController.abort('sibling_error')` only triggers when a Bash tool errors; Read/WebFetch and other concurrency-safe tools are unaffected. This isn't because "read-only tools can't fail" — it's because Bash is the **only tool with implicit side-effect chains**: subsequent Bash commands may depend on the previous command's result (directory existence, environment variable settings, etc.). Other tools' failures don't produce this kind of "ghost follow-up," so there's no need for collateral damage.

2.9.3 runToolUse's "Eight-Phase Execution Pipeline"

The 8-phase design of `runToolUse` embodies the engineering philosophy: **each phase is observable, intervenable, and recoverable**.

Key design: Phase 3's speculative execution

```
// Phase 3: parallel with Phase 2/4
if (tool.name === BASH_TOOL_NAME) {
  startSpeculativeClassifierCheck(command) // Start classifier check early
}
```

The classifier check for Bash commands (determining whether the command is dangerous) starts **in parallel** with permission checks and hook execution. When the permission dialog pops up, the classifier may have already been running for hundreds of milliseconds. This "speculative early start" significantly reduces perceived wait time.

Note

Pragmatic trade-off of the three-layer security architecture: BashTool's security checks aren't "pick one," but layered superposition — regex in `bashSecurity.ts` (dangerous patterns like process substitution, heredocs) filters the most obvious attacks at the outermost layer at minimum cost; TreeSitter AST analysis in `utils/bash/ast.ts` identifies covert techniques that bypass high-risk commands (like `zmodload`, `ztcp`) through string concatenation in the second layer; the third layer is the ML classifier's slow inference. This "fast→slow→accurate" funnel structure is a classic engineering trade-off: **don't use expensive tools to handle problems that can be filtered out by cheap tools**.

Key design: Phase 5's permission decision resolution chain

```
const resolved = await resolveHookPermissionDecision(
  hookPermissionResult, tool, processedInput, toolUseContext, canUseTool, ...
)
if (resolved.decision.behavior !== 'allow') {
  return yield* handlePermissionDenied(...)
}
```

Permission decisions go through **Hook → Rule Check → Pattern Check** three layers; each layer may change the decision, but the final rules (`deny`/`ask`) have the highest priority. This is consistent with the invariant: "Hook's allow does not bypass deny rules in settings.json."

2.9.4 Tool Pool's "Prompt Cache Friendly" Design

```
export function assembleToolPool(permissionContext, mcpTools): Tools {
  const builtInTools = getTools(permissionContext)
  const allowedMcpTools = filterToolsByDenyRules(mcpTools, permissionContext)
  // Sort by name to ensure prompt cache stability
  return uniqBy(
    [...builtInTools].sort(byName).concat(allowedMcpTools.sort(byName)),
    'name',
  )
}
```

Why sort the tool pool?

Claude Code uses Anthropic API's **Prompt Cache** mechanism — identical prompt prefixes are cached and reused. When the tool list is part of the system prompt, if the tool order is non-deterministic (different on each startup), it causes cache misses.

By enforcing alphabetical ordering with `.sort(byName)`:

- Built-in tools: `Agent < Bash < ... < WebSearch`
- MCP tools: appended in alphabetical order as well
- **Result**: Any session with the same tool set shares one cache

MCP tools are always after built-in tools, and via `uniqBy`, built-in tools take priority for same-name conflicts — this is a security boundary: a user's MCP server should not be able to override built-in tool behavior.

Note

Concurrency safety judgment: Answer only after schema parsing: `isConcurrencySafe` is not a static boolean on the tool definition, but a **dynamic method called only after the input is successfully parsed by the Zod Schema**. This means the same tool can change its concurrency behavior based on input — for example, grep is concurrency-safe when read-only, but may not be when it has special flags containing write operations. This design attributes "side effects of an operation" to the **specific call** rather than the tool type, which is a deeper application of the principle "metadata-driven behavior control": **behavior is determined by data, not by identity**.

2.9.5 Error Handling Design: Don't Discard, Tombstone

```
// In query.ts's streaming fallback path
if (reason === 'streaming_fallback') {
  return createUserMessage({
    content: [{
      type: 'tool_result',
      content: '<tool_use_error>Streaming fallback - tool execution discarded</tool_use_error>',
      is_error: true,
      tool_use_id: toolUseId,
    }],
    ...
  })
}
```

Note the use of "tombstone" rather than direct discard. When streaming fallback occurs (model switch, etc.), in-flight tool results are marked as "abandoned" rather than completely gone. This is because:

- Streaming response tool_use blocks correspond to tool_results by **positional index**
- If a tool_result is directly deleted, subsequent tool_use indices will be misaligned
- Using a tombstone error message as a placeholder preserves index integrity

2.9.6 Dual Value of Zod Schema: Runtime Safety + LLM Prompts

```
const parsedInput = tool.inputSchema.safeParse(input)
if (!parsedInput.success) {
  let errorContent = formatZodValidationError(tool.name, parsedInput.error)
  // Additionally add hint about schema not sent
  const schemaHint = buildSchemaNotSentHint(tool, messages, tools)
  if (schemaHint) {
    errorContent += schemaHint
  }
}
```

The tool's Zod schema **serves two purposes simultaneously**:

1. **Runtime type safety**: LLM-generated arguments are validated by `safeParse`
2. **LLM's tool understanding**: schema is sent to the API, LLM uses it to understand the tool's input format

When the model doesn't receive a deferred tool's schema, Claude Code **tells the model how to load the tool in the error message** (`buildSchemaNotSentHint`). This is a self-healing hint mechanism — the model can self-correct behavior after receiving the error.

Note

Trust boundary externalization: MCPTool's zero-validation philosophy: `MCPTool`'s inputSchema is `z.object({}).passthrough()` — it accepts any JSON and passes it through to the MCP server unchanged, with no local validation. This may seem like an "unsafe" design, but it's actually a kind of **architectural trust externalization**: Claude Code delegates tool safety verification responsibility to the MCP server itself. This design aligns with the decentralized nature of the MCP protocol — each MCP server has complete semantic understanding of its own tools, and Claude Code shouldn't (and can't) verify on its behalf. Built-in tools use Zod for type safety; for MCP tools, they choose "pass through but don't understand," which is pragmatic **awareness of capability boundaries**.

2.10 Tool Execution Sequence Diagram (Supplementary)

```
LLM streaming output tool_use block
    │
    ▼
StreamingToolExecutor.addTool()
    │
    ├── [Concurrency-safe?] ──yes──→ canExecuteTool() ──→ Execute immediately
    │                                        │
    │                                ┌──────────────────┐
    │                                ▼                  ▼
    │                        Collecting results...      Collecting results...
```

```
    │                          │                      │
    │                          ▼                      ▼
    │                     Any errors?             All succeeded
    │                    (Bash only)                  │
    │                          │                      │
    │                          ▼                      ▼
    │              siblingAbortController          status='completed'
    │              hasErrored = true                     │
    │                          │                      ▼
    │                          ▼               yield results (by enqueue order)
    │              Notify other concurrent tools to cancel
    │                          │
    └── [Not concurrency-safe?] ──no──→ Wait for all tools in queue to complete
                                  │
                                  ▼
                          Execute subsequent tools serially
```

2.8 Tool Constants and Access Control

File: `constants/tools.ts`

```
// === Tools never exposed to sub-agents ===
export const ALL_AGENT_DISALLOWED_TOOLS = new Set([
  TASK_OUTPUT_TOOL_NAME,       // Prevent reading agent output
  EXIT_PLAN_MODE_V2_TOOL_NAME, // Prevent exiting plan mode
  ENTER_PLAN_MODE_TOOL_NAME,  // Prevent entering plan mode
  // AgentTool: unless ant user (supports nested agents)
  ...(process.env.USER_TYPE === 'ant' ? [] : [AGENT_TOOL_NAME]),
  ASK_USER_QUESTION_TOOL_NAME, // Prevent asking users questions
  TASK_STOP_TOOL_NAME,         // Prevent stopping tasks
  WORKFLOW_TOOL_NAME,          // Prevent recursive workflow execution (if enabled)
])

// === Tools available to async agents (background) ===
export const ASYNC_AGENT_ALLOWED_TOOLS = new Set([
  FileReadTool, WebSearchTool, TodoWriteTool, GrepTool, WebFetchTool,
  GlobTool, BashTool, FileEditTool, FileWriteTool, NotebookEditTool,
  SkillTool, SyntheticOutputTool, ToolSearchTool,
  EnterWorktreeTool, ExitWorktreeTool,
])

// === Tools for in-process teammate only ===
export const IN_PROCESS_TEAMMATE_ALLOWED_TOOLS = new Set([
  TaskCreateTool, TaskGetTool, TaskListTool, TaskUpdateTool,
  SendMessageTool,
  ...(AGENT_TRIGGERS ? [CronCreateTool, CronDeleteTool, CronListTool] : []),
])

// === Coordinator mode tools ===
export const COORDINATOR_MODE_ALLOWED_TOOLS = new Set([
  AgentTool, TaskStopTool, SendMessageTool, SyntheticOutputTool,
])

// === Simple mode ===
// REPL mode: REPLTool + TaskStopTool + SendMessageTool only
// Non-REPL: Bash + FileRead + FileEdit only
```

---

3. Agent System

3.1 AgentTool Overview

`AgentTool` is the core of the entire Agent system, accounting for approximately 125K tokens of implementation code, supporting multiple Agent launch modes.

3.2 Supported Agent Types and Spawn Modes

AgentTool automatically determines which mode to launch based on input parameters:

Spawn Mode	Decision Condition	Description
Fresh Subagent	`subagent_type` has a value	Find specified type from Agent registry and launch
Fork	`subagent_type` empty + FORK_AGENT feature enabled	Implicit context inheritance, shared cache, no independent subprocess
Teammate	Both `team_name` and `name` have values	Multi-Agent team member (tmux/in-process/remote)
General Purpose	`subagent_type` empty + old path	Fall back to `GENERAL_PURPOSE_AGENT`

3.3 Input/Output Schema

```
// Base Schema (always present)
const baseInputSchema = z.object({
  description:  z.string(),           // Short task description (3-5 words)
  prompt:        z.string(),           // Task for agent to complete
  subagent_type: z.string().optional(), // Agent type selector
  model: z.enum(['sonnet', 'opus', 'haiku']).optional(),  // Model override
  run_in_background: z.boolean().optional()  // Async flag
})

// Full Schema (multi-Agent / Teammate mode)
const fullInputSchema = baseInputSchema.extend({
  name:      z.string().optional(),     // Name used for SendMessage routing
  team_name: z.string().optional(),     // Team name
  mode:      permissionModeSchema().optional(),  // Permission mode
  isolation: z.enum(['worktree', 'remote']).optional(),  // Filesystem isolation
  cwd:       z.string().optional()      // Working directory override
})

// === Output Types ===

// Sync output (foreground completion)
{ status: 'completed', agentId, agentType, content, totalToolUseCount,
  totalDurationMs, totalTokens, usage, prompt }

// Async output (background launch)
{ status: 'async_launched', agentId, description, prompt,
  outputFile, canReadOutputFile }
```

```
// Teammate output
{ status: 'teammate_spawned', prompt, teammate_id, agent_id,
  sessionId?, paneId?, windowId?, windowName? }

// Remote output (CCR)
{ status: 'remote_launched', taskId, sessionUrl, description,
  prompt, outputFile }

// Resume output
{ status: 'resumed', agentId, description, prompt, outputFile }
```

3.4 Sync vs. Async Agent Differences

Dimension	Sync Agent	Async Agent
Waiting method	`Promise.race(nextMessagePromise, backgroundPromise)`	Directly returns `async_launched`
AbortController	Shared with parent agent	Independent (ESC cancel doesn't stop background agent)
Auto-backgrounding	Auto-backgrounds after `getAutoBackgroundMs()` (default 120s)	Directly registered as background task
Registration method	`registerAgentForeground()`	`registerAsyncAgent()`
Lifecycle	Parent agent polls `backgroundPromise`	`runAsyncAgentLifecycle()` manages independently
Cache sharing	Shares `PromptCache` with parent	Independent cache (`cache.eviction_hint`)

3.5 Worktree Isolation

```
// When isolation: "worktree"
const slug = `agent-${earlyAgentId.slice(0, 8)}`
worktreeInfo = await createAgentWorktree(slug)

// Agent runs in an independent Git worktree; file changes don't affect main branch
// Post-completion processing logic:
if (headCommit && !hasWorktreeChanges(worktreePath, headCommit)) {
  // No changes: auto-delete worktree, save space
  await removeAgentWorktree(worktreePath, worktreeBranch, gitRoot)
}
// Has changes: keep worktree, return branch info for user to review
```

3.6 Agent Definition File Format

Agent definitions support loading from Markdown files or YAML frontmatter (`loadAgentsDir.ts`):

```
---
name: my-agent                    # Required - Agent identifier
description: When to use this agent      # Required - Displayed to parent agent
color: AgentColorName             # Optional - UI color
model: opus-4 | inherit           # Optional - Model override
effort: xs | sm | md | lg | xl    # Optional - Effort level
permissionMode: plan | acceptEdits  # Optional - Permission mode
tools:                            # Optional - Tool allowlist
  - '*'
  - Bash
  - Read
disallowedTools:                  # Optional - Tool denylist
  - Write
  - TaskOutput
maxTurns: 50                      # Optional - Max turn limit
background: true                  # Optional - Always run async
initialPrompt:前置提示词          # Optional - Appended to first user input
skills:                           # Optional - Preloaded skills
  - skill1
  - skill2
memory: user | project | local    # Optional - Persistent memory scope
isolation: worktree | remote      # Optional - Filesystem isolation
mcpServers:                        # Optional - MCP server configuration
  - slack                          # Reference: find existing config
  - myServer:                      # Inline: dynamic config
      command: npx
      args: ['-y', '@modelcontextprotocol/server-filesystem', '/tmp']
hooks:                            # Optional - Session hooks
  Stop:
    - hook: my-hook
---
# System prompt body (Agent's core instructions)
You are an agent specialized in handling...
```

Agent Definition Type System

```
// Base type
type BaseAgentDefinition = {
  agentType: string
  whenToUse: string
  tools?: string[]
  disallowedTools?: string[]
  skills?: string[]
  mcpServers?: AgentMcpServerSpec[]
  hooks?: HooksSettings
  color?: AgentColorName
  model?: string
  effort?: EffortValue
  permissionMode?: PermissionMode
  maxTurns?: number
  filename?: string
  baseDir?: string
  requiredMcpServers?: string[]   // Required MCP server connections
  background?: boolean
  initialPrompt?: string
  memory?: AgentMemoryScope
  isolation?: 'worktree' | 'remote'
  omitClaudeMd?: boolean           // Skip CLAUDE.md
}
```

```
// Built-in Agent (dynamic prompts)
type BuiltInAgentDefinition = BaseAgentDefinition & {
  source: 'built-in'
  baseDir: 'built-in'
  callback?: () => void
  getSystemPrompt: (params: { toolUseContext }) => string
}

// Custom Agent (user/project/policy settings)
type CustomAgentDefinition = BaseAgentDefinition & {
  getSystemPrompt: () => string
  source: SettingSource  // 'user' | 'project' | 'policy'
}

// Plugin Agent
type PluginAgentDefinition = BaseAgentDefinition & {
  getSystemPrompt: () => string
  source: 'plugin'
  plugin: string
}

// MCP server spec
type AgentMcpServerSpec =
  | string                              // Reference: "slack"
  | { [name: string]: McpServerConfig } // Inline: { myServer: {...} }
```

Agent Loading Priority (later overrides earlier)

```
1. Built-in Agent (built-in)
2. Plugin Agent (plugin)
3. User settings Agent (~/.claude/settings.json)
4. Project settings Agent (.claude/settings.json)
5. Flag settings Agent (--agent-dir)
6. Policy settings Agent (policy settings)
```

3.7 Built-in Agents

Agent	File	Description
`general-purpose`	`built-in/generalPurposeAgent.ts`	Default general agent (`tools: ['*']`)
`plan`	`built-in/planAgent.ts`	Plan-type agent
`Explore`	`built-in/exploreAgent.ts`	Codebase exploration agent (One-shot, skips trailer)
`Plan`	`built-in/planAgent.ts`	Plan agent (One-shot, skips trailer)
`verification`	`built-in/verificationAgent.ts`	Code verification agent
`claude-code-guide`	`built-in/claudeCodeGuideAgent.ts`	Claude Code usage guide
`statusline-setup`	`built-in/statuslineSetup.ts`	Status bar setup agent

One-shot Built-in Agent Optimization: Explorer and Plan Agent set `ONE_SHOT_BUILTIN_AGENT_TYPES`, skipping agentId/SendMessage/usage trailer, saving approximately 135 tokens × ~34 million Explore calls per week.

3.8 runAgent Execution Engine

```
async function* runAgent({
  agentDefinition,
  promptMessages,
  toolUseContext,
  canUseTool,
  isAsync,
  forkContextMessages,
  querySource,
  override,
  model,
  maxTurns,
  availableTools,
  allowedTools,
  onCacheSafeParams,
  contentReplacementState,
  useExactTools,
  worktreePath,
  description,
  transcriptSubdir,
  onQueryProgress,
}): AsyncGenerator<Message, void>
```

Agent ID creation:

```
const agentId = override?.agentId ?? createAgentId()
// Format: UUID v4
```

Context resolution:

```
const [baseUserContext, baseSystemContext] = await Promise.all([
  override?.userContext ?? getUserContext(),
  override?.systemContext ?? getSystemContext(),
])

// Explore/Plan Agent can opt to skip CLAUDE.md
const shouldOmitClaudeMd = agentDefinition.omitClaudeMd && !override?.userContext
```

MCP server initialization:

```
// Agent can connect its own MCP servers (independent of parent agent)
async function initializeAgentMcpServers(
  agentDefinition,
  parentClients: MCPServerConnection[],
): Promise<{ clients, tools, cleanup }> {
  // Type 1: String reference → find existing MCP config
  // Type 2: Inline definition → create new dynamic MCP client (independent cleanup)
```

```
}
```

Query loop:

```
for await (const message of query({
  messages: initialMessages,
  systemPrompt: agentSystemPrompt,
  userContext: resolvedUserContext,
  systemContext: resolvedSystemContext,
  canUseTool,
  toolUseContext: agentToolUseContext,
  maxTurns: maxTurns ?? agentDefinition.maxTurns,
})) {
  if (message.type === 'attachment') {
    if (message.attachment.type === 'max_turns_reached') break
    yield message; continue
  }
  if (isRecordableMessage(message)) {
    await recordSidechainTranscript([message], agentId, lastRecordedUuid)
    yield message
  }
}
```

Cleanup (finally block):

```
finally {
  await mcpCleanup()                      // Agent-specific MCP servers
  clearSessionHooks(rootSetAppState, agentId)
  cleanupAgentTracking(agentId)            // Prompt cache tracking
  agentToolUseContext.readFileState.clear() // File state cache
  unregisterPerfettoAgent(agentId)         // Perfetto tracking
  clearAgentTranscriptSubdir(agentId)      // Transcript subdirectory
  // Cleanup todo items
  rootSetAppState(prev => {
    const { [agentId]: _, ...todos } = prev.todos
    return { ...prev, todos }
  })
  killShellTasksForAgent(agentId)          // Background Bash tasks
}
```

3.9 Fork Subagent Feature

Fork is a unique Agent mode in Claude Code:

- **Implicit context inheritance**: Parent agent's message history is automatically passed to Fork, no manual passing needed
- **Shared cache**: `forkContextMessages` lets Fork share prompt cache benefits with parent
- **Fork guidance**: Auto-generates Fork's system prompt and task description (generated by Fork rules in `prompt.ts`)
- **Recursion protection**: Counter prevents infinite Fork nesting
- **Use cases**:
  - Open-ended research → launch parallel Forks
  - Complex tasks requiring more than 2 edits → Fork to execute
  - "Like a smart colleague who just walked in" style task descriptions

3.10 Agent Memory System

```
// Memory scope
type AgentMemoryScope = 'user' | 'project' | 'local'

// Directory structure
function getAgentMemoryEntrypoint(agentType, scope): string {
  // user:   <memoryBase>/agent-memory/<agentType>/MEMORY.md
  // project: <cwd>/.claude/agent-memory/<agentType>/MEMORY.md
  // local:  <cwd>/.claude/agent-memory-local/<agentType>/MEMORY.md
  //         or remote mount path
}

function loadAgentMemoryPrompt(agentType, scope): string {
  // Returns different loading prompts based on scope
  // user:   General learning (cross-project)
  // project: Project-specific (shared via VCS)
  // local:  Project/machine-specific (not in VCS)
}

// Snapshot system
// agent-memory-snapshots/<agentType>/snapshot.json
// Periodic snapshots for recovery and sync
```

3.11 AgentTool Constants

```
export const AGENT_TOOL_NAME = 'Agent'
export const LEGACY_AGENT_TOOL_NAME = 'Task'  // Backward-compatible alias

export const VERIFICATION_AGENT_TYPE = 'verification'

// One-shot built-in Agent types (skip trailer)
export const ONE_SHOT_BUILTIN_AGENT_TYPES: ReadonlySet<string> = new Set(['Explore', 'Plan'])
```

3.12 Agent System Design Philosophy Analysis

3.12.1 The Essence of Fork: Zero-overhead Context Fork

The document describes Fork as "implicit context inheritance," but a more precise understanding is: **Fork is a "Copy-on-Write" conversation snapshot**.

```
// Fork subagent's message history injected via forkContextMessages
const forkContextMessages = override?.forkContextMessages ?? forkParentMessages(...)
// forkParentMessages takes key context from the end of parent agent's messages
```

Key differences between Fork and truly launching a new agent:

Dimension	Fresh Subagent	Fork
Context passing	Manual injection (explicit pass every time)	Auto-inheritance (fork rule generator auto-builds)
Prompt Cache	No sharing (new prompt independently computes cache)	Shares parent's cache benefit
Exit method	Via AbortController	Auto-sends notification on completion
Use case	Long-running independent tasks	Short exploratory tasks

Fork's cache sharing is bidirectional: Fork's runtime API calls also contribute cache benefit to parent agent's subsequent calls. This creates a "community effect" — all agents in the same turn collectively build the cache.

3.12.2 AsyncLocalStorage: The Standard Answer for In-Process Isolation

When Claude Code runs multiple agents **in the same process** (In-Process Teammate, Fork subagent), the core problem is: Node.js's `async/await` "yields" execution to other coroutines at await points, and traditional module-level variables get overwritten.

Claude Code's solution:

```
const agentContextStorage = new AsyncLocalStorage<AgentContext>()

export function runWithAgentContext<T>(context: AgentContext, fn: () => T): T {
  return agentContextStorage.run(context, fn)
}

export function getAgentContext(): AgentContext | undefined {
  return agentContextStorage.getStore()
}
```

How AsyncLocalStorage works:

- Each `run()` call creates a "storage slot" bound to the current async call chain
- No matter how many awaits are in the call chain, this slot is always accessible
- Subagent's `runAgent()` executes within `runWithAgentContext()`
- `getAgentContext()` can access the correct agent context across any async boundary

What does this solve?

```
// AppState is globally shared and contains all task state for the current agent
// If two agents modify AppState simultaneously, they overwrite each other
// Through AsyncLocalStorage, each agent accesses its "own slot's" context
```

3.12.3 One-shot Agent Optimization: Minimal Protocol for High-frequency Calls

Explore and Plan Agent set `ONE_SHOT_BUILTIN_AGENT_TYPES`, skipping:

- agentId trailer (~30 chars)
- SendMessage routing trailer (~50 chars)
- usage stats trailer (~55 chars)

Total approximately 135 chars × per Explore call. As the document states, ~34 million Explore calls per week means:

- Weekly savings: 34M × 135 chars ≈ **4.6GB of token traffic** (at avg 4 chars/token)
- Quality guarantee: what's skipped is "routing information," not "result content," so agent output quality is unaffected

This is a typical **idempotent operation optimization**: for agents with predictable behavior, knowing in advance they won't return continued communication, you can directly skip the protocol handshake.

3.12.4 Engineering Significance of Worktree Isolation

Worktree isolation is not just filesystem isolation; it solves a real engineering problem:

```
Problem: Multiple agents simultaneously operating on different files of the same repo
Risk: Git operation conflicts (bad timing for merge/rebase)

Solution: Each agent runs in an independent Git worktree
          worktree is Git's natively supported feature
          No need for Docker/namespace or other heavy isolation
```

The post-completion processing logic (auto-cleanup when no changes) further reflects **resource management awareness**: long-running Claude Code sessions accumulate many worktrees; without timely cleanup, `git worktree list` gets slower.

```
export const AGENT_TOOL_NAME = 'Agent'
export const LEGACY_AGENT_TOOL_NAME = 'Task'  // Backward-compatible alias

export const VERIFICATION_AGENT_TYPE = 'verification'

// One-shot built-in Agent types (skip trailer)
export const ONE_SHOT_BUILTIN_AGENT_TYPES: ReadonlySet<string> = new Set(['Explore', 'Plan'])
```

---

4. Multi-Agent Collaboration and Teams

4.1 Team Tools

Tool	File	Responsibility
`TeamCreateTool`	`TeamCreateTool.ts`	Create Agent team
`TeamDeleteTool`	`TeamDeleteTool.ts`	Delete team
`SendMessageTool`	`SendMessageTool.ts`	Send messages to teammates (peer-to-peer/broadcast/structured protocol)

4.2 Swarm Backend System (`utils/swarm/`)

Claude Code supports three teammate execution backends:

TmuxBackend

```
export class TmuxBackend implements PaneBackend {
  readonly type = 'tmux'
  readonly displayName = 'tmux'
  readonly supportsHideShow = true

  // Split in existing tmux window (leader on left, teammate on right)
  createTeammatePaneWithLeader(name, color): Promise<CreatePaneResult>
```

```
  // Create independent swarm session
  createTeammatePaneExternal(name, color): Promise<CreatePaneResult>
  // Rebalance layout (leader 30% / teammate 70% or tiled)
  rebalancePanesWithLeader(hasLeader): Promise<void>
  rebalancePanesTiled(): Promise<void>
}
```

- Leader pane fixed at 30% width; teammate pane shares remaining 70%
- Uses socket isolation for external sessions
- Supports hide/show pane
- Uses pane creation lock during parallel spawn

ITermBackend

```
export class ITermBackend implements PaneBackend {
  readonly type = 'iterm2'
  readonly displayName = 'iTerm'
  readonly supportsHideShow = false  // No hide/show support

  // Uses it2 CLI: it2 session split / it2 session run / it2 session close
  // Vertical split (leader on left, teammates stacked on right)
}
```

- Communicates with iTerm2 via `it2` CLI
- Vertical split layout
- Dead session periodic cleanup

InProcessBackend

```
export class InProcessBackend implements TeammateExecutor {
  readonly type = 'in-process'
  // Always available (no external dependencies)
  // Uses AsyncLocalStorage for context isolation
  // Shares API client and MCP connections
  // File mailbox communication
  // AbortController termination
}
```

- Zero overhead, suitable for lightweight tasks
- All teammates in the same Node.js process
- Context isolation via AsyncLocalStorage

4.3 Teammate Spawn Flow (`tools/shared/spawnMultiAgent.ts`)

```
async function handleSpawn(input, context) {
  // 1. Model resolution
  const resolvedModel = resolveTeammateModel(input.model, leaderModel)

  // 2. Generate team name (unique)
  const teamName = input.team_name ?? generateTeamName()

  // 3. Generate Agent ID (format: "name@teamName")
  const agentId = formatAgentId(sanitizedName, teamName)

  // 4. Assign color (multi-Agent UI display)
  const color = await assignTeammateColor()

  // 5. Create pane/window
  const backend = await detectAndGetBackend()
  const paneResult = await backend.createTeammatePane(name, color)

  // 6. Build spawn command (inherits CLI flags)
  const cmd = buildSpawnCommand({
    model: resolvedModel,
    permissionMode,  // bypassPermissions → --dangerously-skip-permissions
    plugins,         // --plugin-dir each inline plugin
    settings,
  })

  // 7. Send to pane
  await backend.sendCommandToPane(paneId, cmd)

  // 8. Register task and team
  registerOutOfProcessTeammateTask(taskState)
  writeTeamFile(teamName, { members: [...prev.members, { agentId, color }] })

  // 9. Send initial prompt
  if (isTmux) {
    await writeToMailbox(name, { from: TEAM_LEAD_NAME, text: prompt, ... }, teamName)
  }
}
```

4.4 SendMessageTool Communication Protocol

Input Schema:

```
{
  to: string,           // "teammateName" | "*" (broadcast) | "uds:<path>" | "bridge:<session>"
  summary?: string,     // 5-10 word preview (UI display)
  message: string | StructuredMessage
}

// Structured message types
type StructuredMessage =
  | { type: 'shutdown_request', reason?: string }
  | { type: 'shutdown_response', request_id: string, approve: boolean, reason?: string }
  | { type: 'plan_approval_response', request_id: string, approve: boolean, feedback?: string }
```

Communication methods:
| Route Target | Mechanism | Description |

|---|---|---|
Same-process teammate	`queuePendingMessage()` → delivered at next tool boundary	Memory queue
tmux teammate	`writeToMailbox()` → disk mailbox file	File polling
iTerm2 teammate	`writeToMailbox()` → disk mailbox file	File polling
Remote teammate	`postInterClaudeMessage()` (CCR bridge)	Anthropic server relay
Unix Domain Socket	`sendToUdsSocket()`	Local socket

TaskNotification XML format:

```
<task-notification>
  <task-id>agentId</task-id>
  <tool-use-id>toolUseId</tool-use-id>
  <output-file>/path/to/output.json</output-file>
  <status>completed|failed|killed</status>
  <summary>Agent "description" completed</summary>
  <result>Final message text</result>
  <usage>
    <total_tokens>N</total_tokens>
    <tool_uses>N</tool_uses>
    <duration_ms>N</duration_ms>
  </usage>
  <worktree>
    <worktree-path>/path</worktree-path>
    <worktree-branch>branch-name</worktree-branch>
  </worktree>
</task-notification>
```

4.5 Shutdown Protocol

```
// Request shutdown
handleShutdownRequest(targetName, reason) {
  const request_id = generateRequestId('shutdown', targetName)
  writeToMailbox(targetName, { type: 'shutdown_request', request_id, reason })
  return { request_id }  // For tracking
}

// Approve shutdown (teammate side)
handleShutdownApproval(request_id) {
  writeToMailbox(TEAM_LEAD_NAME, { type: 'shutdown_response', request_id, approve: true })
  abortController.abort()  // In-process: abort immediately
  setImmediate(() => process.exit(0))  // Out-of-process: graceful exit
}

// Reject shutdown
handleShutdownRejection(request_id, reason) {
  writeToMailbox(TEAM_LEAD_NAME, { type: 'shutdown_response', request_id, approve: false, reason })
}
```

4.6 AgentContext and AsyncLocalStorage

```
// Agent context type
type SubagentContext = {
  agentId: string
  parentSessionId?: string
  agentType: 'subagent'
  subagentName?: string  // "Explore", "Bash"
  isBuiltIn?: boolean
  invokingRequestId?: string
  invocationKind?: 'spawn' | 'resume'
}

type TeammateAgentContext = {
  agentId: string       // "agentName@teamName"
  agentName: string
  teamName: string
  agentColor?: string
  planModeRequired: boolean
  parentSessionId: string
  isTeamLead: boolean
  agentType: 'teammate'
}

// Isolation mechanism
const agentContextStorage = new AsyncLocalStorage<AgentContext>()

export function runWithAgentContext<T>(context: AgentContext, fn: () => T): T
export function getAgentContext(): AgentContext | undefined

// Why use AsyncLocalStorage?
// - Isolate concurrent agents' contexts in the same process
// - AppState is shared; direct assignment would be overwritten
// - Especially critical for Ctrl+B backgrounded agents
```

---

5. Task Framework

5.1 Task Type System

```
LocalAgentTask/       ── Background agent task runner
  LocalAgentTask.tsx
  framework.ts         ← Generic task registration/tracking infrastructure
  framework.js

RemoteAgentTask/      ── Remote agent execution (via CCR)
  RemoteAgentTask.tsx

InProcessTeammateTask/ ── In-process teammate task
  InProcessTeammateTask.tsx

LocalShellTask/       ── Shell subprocess task
  LocalShellTask.tsx

DreamTask/            ── Autonomous exploration task
```

```
  DreamTask.tsx
```

5.2 LocalAgentTask Complete State Machine

```
type LocalAgentTaskState = TaskStateBase & {
  type: 'local_agent'
  agentId: string
  prompt: string
  selectedAgent?: AgentDefinition
  agentType: string
  model?: string
  abortController?: AbortController
  unregisterCleanup?: () => void
  error?: string
  result?: AgentToolResult
  progress?: AgentProgress          // Real-time progress
  retrieved: boolean
  messages?: Message[]              // Accumulated messages (appended as streaming)
  lastReportedToolCount: number
  lastReportedTokenCount: number
  isBackgrounded: boolean
  pendingMessages: string[]         // Pending delivery message queue
  retain: boolean                   // UI retention (prevents GC)
  diskLoaded: boolean               // Sidechain JSONL already read
  evictAfter?: number               // GC timestamp
}
```

5.3 Core Operations

Register Background Agent

```
function registerAsyncAgent(
  taskId: string,
  prompt: string,
  agentDefinition: AgentDefinition,
  parentAbortSignal?: AbortSignal,
  autoBackgroundMs?: number,
): Promise<AgentToolResult>

// Flow:
// 1. Create AbortController (optionally inherit parent signal)
// 2. Set isBackgrounded = true
// 3. Register cleanup handler
// 4. registerTask(taskState, setAppState) ← added to AppState.tasks
// 5. Return async result Promise
```

Foreground Agent (can auto-background)

```
function registerAgentForeground(
  taskId, prompt, agentDefinition, parentAbortSignal, autoBackgroundMs
): { backgroundSignal: Promise<void> }

// Flow:
// 1. Register as non-background task (isBackgrounded = false)
// 2. Start agent
// 3. Wait for backgroundSignal to resolve
//    - Timeout auto-backgrounds: backgroundAgentTask()
//    - Task completes: resolve
//    - User ESC: move to background
```

Backgrounding

```
function backgroundAgentTask(taskId, setAppState) {
  // Update state
  setTaskState<LocalAgentTaskState>(taskId, setAppState, task => ({
    ...task,
    isBackgrounded: true,
  }))
  // Resolve backgroundSignal → interrupt agent loop
  backgroundSignal.resolve()
  // Send notification to UI
}
```

Kill Task

```
function killAsyncAgent(taskId, setAppState) {
  abortController.abort()
  unregisterCleanup()
  setTaskState(taskId, setAppState, task => ({
    ...task,
    status: 'killed',
    endTime: Date.now(),
    evictAfter: Date.now() + PANEL_GRACE_MS,  // Keep for 5 minutes for UI review
  }))
  triggerDiskOutputEviction(taskId)  // Trigger disk output eviction
}
```

5.4 Progress Tracking

```
type AgentProgress = {
  toolUseCount: number
  tokenCount: number
  lastActivity?: ToolActivity       // Most recent activity
  recentActivities?: ToolActivity[]  // Last 5 activities
  summary?: string                   // 1-2 sentence summary
}
```

```
type ToolActivity = {
  activityDescription: string        // "Editing auth/login.ts"
  isSearchActivity: boolean
  isReadActivity: boolean
  timestamp: number
}

// Update progress from assistant message
function updateProgressFromMessage(tracker, message, resolver?, tools?) {
  // Count tokens (input cumulative, output sum)
  // Classify tool activity (search vs read)
  // Maintain last 5 activities
}

// Periodic update (UI refresh)
updateAgentProgress(taskId, progress, setAppState)
updateAgentSummary(taskId, summary, setAppState)  // → emit SDK progress event
```

5.5 Message Queue and Delivery

```
// Inter-task message queue (appended as streaming)
queuePendingMessage(taskId, msg, setAppState)
  → pendingMessages.push(msg)

// Drain at tool boundary
drainPendingMessages(taskId, getAppState, setAppState)
  → take pendingMessages, yield to parent agent

// Append directly to messages (immediate display)
appendMessageToLocalAgent(taskId, message, setAppState)
```

5.6 Lifecycle Hooks

```
// Pre-spawn
executeSubagentStartHooks(agentId, agentType, abortSignal)
  → yield additionalContexts appended to initialMessages

// Register Session Hooks during agent run
registerFrontmatterHooks(
  rootSetAppState, agentId,
  agentDefinition.hooks,
  `agent '${agentDefinition.agentType}'`,
  true,  // isAgent: convert Stop hooks to SubagentStop
)

// Post-completion
clearSessionHooks(rootSetAppState, agentId)
clearInvokedSkillsForAgent(agentId)  // Cleanup skill tracking
clearDumpState()                      // Cleanup dump state
```

---

6. Coordinator Mode

6.1 Overview

Coordinator mode is Claude Code's **multi-Agent orchestration** solution: the main agent acts as Coordinator, decomposing tasks to multiple parallel Worker agents.

6.2 Complete System Prompt Structure

`getCoordinatorSystemPrompt()` returns the complete Coordinator prompt, containing:

Section 1 - Role Definition

```
You are Claude Code, an AI assistant that orchestrates software engineering tasks across multiple workers.
Your job is to:
- Help the user achieve their goal
- Direct workers to research, implement and verify code changes
- Synthesize results and communicate with the user
- Answer questions directly when possible
```

Section 2 - Tool Definitions

- `AgentTool` (spawn workers)
- `SendMessageTool` (continue existing workers)
- `TaskStopTool` (stop workers)
- `subscribe_pr_activity` / `unsubscribe_pr_activity` (GitHub PR events)

Section 3 - Worker Result Format

Task notifications arrive as `<task-notification>` XML, containing `<task-id>`, `<status>`, `<summary>`, `<result>`, `<usage>` tags.

Section 4 - Workers (Worker Agent definitions)

```
// Worker toolset: exclude INTERNAL_WORKER_TOOLS
const INTERNAL_WORKER_TOOLS = new Set([
  TEAM_CREATE_TOOL_NAME,   // TeamCreateTool
  TEAM_DELETE_TOOL_NAME,   // TeamDeleteTool
  SEND_MESSAGE_TOOL_NAME,  // SendMessageTool
  SYNTHETIC_OUTPUT_TOOL_NAME, // SyntheticOutputTool
])

// SIMPLE mode (lightweight)
workerTools = [BASH_TOOL_NAME, FILE_READ_TOOL_NAME, FILE_EDIT_TOOL_NAME].sort()

// DEFAULT mode (full)
workerTools = Array.from(ASYNC_AGENT_ALLOWED_TOOLS)
  .filter(name => !INTERNAL_WORKER_TOOLS.has(name))
  .sort()
```

Section 5 - Task Workflow

Phase	Executor	Purpose
Research	Workers (parallel)	Investigate codebase
Synthesis	Coordinator	Read findings, create specs
Implementation	Workers	Implement changes per spec
Verification	Workers	Prove code works correctly

Section 6 - Concurrency Rules

- Read-only tasks: freely parallel
- Write-intensive tasks: one group of files per Worker at a time
- Verification tasks: can parallel with implementation on different areas

Section 7 - Spawn vs. Continue Decision Matrix

Scenario	Decision
Overlapping files	Continue (SendMessageTool)
Broad research, narrow implementation	Spawn new
Correction/failure	Continue
Verify someone else's code	Spawn new
Completely wrong direction	Spawn new
Unrelated tasks	Spawn new

6.3 Spawn vs. Continue Decision Principles

Core principle: Don't delegate understanding; synthesize yourself first, then give Workers specific instructions.

```
When Spawning:
  - Must include specific file paths, line numbers, error messages
  - Include "purpose statement" for calibration
  - Include information you already know (don't make Worker repeat research)

When Continuing:
  - Provide correction or additional context
  - Continue when files overlap (already have context)
```

6.4 Tool Constraints Summary

Role	Available Tools	Unavailable Tools
Coordinator	Agent, SendMessage, TaskStop, SyntheticOutput	All built-in tools
Worker (SIMPLE)	Bash, Read, Edit + MCP	Agent, SendMessage, TeamCreate, TeamDelete, etc.
Worker (DEFAULT)	ASYNC_AGENT_ALLOWED_TOOLS minus INTERNAL_WORKER_TOOLS	Same as above + most built-in tools

---

7. Query Engine

7.1 QueryEngine Class

File: `QueryEngine.ts`

```
export class QueryEngine {
  private config: QueryEngineConfig
  private mutableMessages: Message[]
  private abortController: AbortController
  private permissionDenials: SDKPermissionDenial[]
  private totalUsage: NonNullableUsage
  private hasHandledOrphanedPermission = false
  private readFileState: FileStateCache
  private discoveredSkillNames = new Set<string>()

  // Core methods
  async *submitMessage(prompt, options?): AsyncGenerator<SDKMessage>
  interrupt(): void
  getMessages(): readonly Message[]
  getReadFileState(): FileStateCache
  getSessionId(): string
  setModel(model: string): void
}
```

7.2 submitMessage Execution Flow

```
async *submitMessage(prompt, options?): AsyncGenerator<SDKMessage> {
  // Phase 1: Setup
  const canUseTool = wrapCanUseTool(originalCanUseTool, {
    onDenial: denial => this.permissionDenials.push(denial)
  })
  const systemPromptParts = await getSystemPromptParts(config)

  // Phase 2: Orphaned permission handling
  if (this.hasHandledOrphanedPermission) {
    yield* yieldPendingPermissions(canUseTool, ...)
  }

  // Phase 3: processUserInput (slash commands, model changes, tool permissions)
  const { modifiedMessages, shouldCompact, isCommand } = await processUserInput(...)

  // Phase 4: Transcript recording
  await recordSidechainTranscript(modifiedMessages, sessionId, ...)

  // Phase 5: Eager flush
  if (isEagerFlush()) {
    await flushSessionStorage(this.sessionId)
  }

  // Phase 6: Query loop
  for await (const message of query({ messages: [...], ...config })) {
    yield { type: 'sdk_message', sessionId, message }  // Wrapped and yielded
  }
```

```
  // Phase 7: Result
  return { type: 'sdk_result', textResult, permissionDenials, totalUsage }
}
```

7.3 query.ts Main Query Loop

File: `query.ts`

```
async function* query(params: QueryParams): AsyncGenerator<Message> {
  const consumedCommandUuids: string[] = []
  const terminal = yield* queryLoop(params, consumedCommandUuids)
  for (const uuid of consumedCommandUuids) {
    notifyCommandLifecycle(uuid, 'completed')
  }
  return terminal
}
```

7.4 queryLoop State Machine

```
type State = {
  messages: Message[]                          // Accumulated messages
  toolUseContext: ToolUseContext
  autoCompactTracking: AutoCompactTrackingState | undefined
  maxOutputTokensRecoveryCount: number         // Retry count
  hasAttemptedReactiveCompact: boolean
  maxOutputTokensOverride: number | undefined
  pendingToolUseSummary: Promise | undefined
  stopHookActive: boolean | undefined
  turnCount: number
  transition: Continue | undefined
}
```

7.5 Turn Loop (while(true))

Each Turn contains the following steps:

```
[1] Increment chain depth (root depth=0)
    ↓
[2] HISTORY_SNIP — trim old messages before limit exceeded
    ↓
[3] Microcompact — cache expensive tool results (lazy boundary)
    ↓
[4] Context Collapse — phased commit (CONTEXT_COLLAPSE feature)
    ↓
[5] Autocompact — check if auto-compression is triggered
    ↓
[6] Blocking limit check — hard block threshold check
    ↓
[7] API Streaming Loop
    ├─ callModel() — streaming API call
    ├─ Streaming fallback — model switch + retry
    ├─ Backfill tool inputs — complete tool inputs
    └─ Withhold recoverable errors — hold PTL/max_output_tokens errors
    ↓
[8] Post-sampling hooks — executePostSamplingHooks
    ↓
[9] Abort handling — consume remaining stream / yield missing blocks
    ↓
[10] Tool use summary — Haiku generates summary (emitToolUseSummaries)
    ↓
[11] Prompt-too-long recovery
    ├─ Context collapse drain
    ├─ Reactive compact
    └─ Escalate or surface error
    ↓
[12] Max output tokens recovery
    ├─ 8k default → 64k escalate
    └─ Multi-turn recovery with resume message
    ↓
[13] Stop hooks — handleStopHooks
    ├─ Blocking errors retry loop
    └─ preventContinuation → return early
    ↓
[14] Token budget — continue or terminate decision
    ↓
[15] Tool execution
    ├─ StreamingToolExecutor.getRemainingResults() (streaming)
    └─ runTools() (non-streaming)
    ↓
[16] Queue drain — process queued commands (task notifications, prompts)
    ↓
[17] Memory prefetch — attach relevant memory files
    ↓
[18] Skill discovery prefetch — attach discovered skills
    ↓
[19] Max turns check — exit if limit exceeded
    ↓
[20] Continue — build next state, continue loop
```

7.6 Key Recovery Mechanisms

Prompt-too-long Recovery (PTL Recovery)

```
// Withhold PTL error until stream ends
isWithheldPTLError(error) → true

// Recovery path:
// 1. Context collapse drain (drain committed content)
// 2. Reactive compact (incremental compression)
// 3. Escalate: surface error if still over limit
```

```
// isRecoverablePTLError detection:
// - Contains 'messages' field
// - Non-structured output
// - Non-Anthropic error code
```

Max Output Tokens Recovery

```
const MAX_OUTPUT_TOKENS_RECOVERY_LIMIT = 3

// Recovery path:
// 1. 8k default → escalate to 64k
// 2. Multi-turn: insert resume message, continue conversation
// 3. After 3 times: give up and return result
```

Streaming Fallback

```
// Triggered when model is unavailable or unsupported
// Strategy: switch model and retry
// For protected thinking models: strip signature blocks
// Orphaned messages are "tombstoned"
```

7.7 Query Engine Design Philosophy Analysis

7.7.1 AsyncGenerator as the Architectural Backbone

Claude Code uses `AsyncGenerator` throughout to pass message streams (`query.ts`, `runAgent.ts`, `runToolUse()`), which is not just a technical choice but the **organizing principle** of the entire architecture.

Why AsyncGenerator?

```
// Traditional callback style
async function query(params, callback: (msg: Message) => void): Promise<void>

// AsyncGenerator style
async function* query(params): AsyncGenerator<Message>
```

Both are functionally equivalent, but AsyncGenerator provides key **composability advantages**:

```
query() ──yield── Message ──yield── Message ──yield── ...
                  |                  |
              Consumed by callee      Caller continues iteration

// The same interface, caller can be:
// - CLI UI (real-time message rendering)
// - SDK wrapper (forward to third party)
// - Test stub (intercept specific messages)
// - Debug tool (record all messages)
```

More importantly, `yield*` (delegated generator) makes pipeline composition natural:

```
// In QueryEngine.submitMessage
for await (const message of query({ messages: [...], ...config })) {
  yield { type: 'sdk_message', sessionId, message }
}
// query()'s generator is "flattened" into submitMessage's generator
// Caller is completely unaware of the intermediate layer
```

Note

The essence of AsyncGenerator is a "stoppable pipeline": each `yield` is a **checkpoint** — the caller can pause, cancel, or reset iteration at any checkpoint. This checkpoint mechanism is the底层 foundation for Claude Code's ESC interruption, sibling abort, and permission dialogs: **not "cutting in" during execution, but gaining control at the generator's next `yield`**. This also explains why "pressing ESC during tool execution" takes effect immediately — the CLI is waiting for the next `yield`; upon receiving `abortController.signal`, it can directly short-circuit remaining iteration without waiting for the current tool to complete.

7.7.2 The 20 Steps of the Turn Loop: Phased State Machine

The 20 steps in the document are a carefully designed state machine, each with clear **boundary conditions and side effects**:

The most insightful steps:

[2] HISTORY_SNIP executes **before** the model call, meaning compression happens at a historical boundary the model "cannot see." This ensures:

- Compressed context is coherent to the model (no half-truncated messages)
- The model is unaware it was compressed

[7] Backfill in the API Streaming Loop:

```
// Complete tool inputs — streaming response may only contain partial parameters
// Claude Code waits here for complete parameters
```

This is the **inverse of streaming processing**: LLM output is streaming, but tool calls need complete parameters.

[11] Prompt-too-long Recovery's delayed strategy:

```
// Error is withheld, not reported immediately
isWithheldPTLError(error) → true
```

This is done to wait for the stream to end, because PTL errors may appear mid-stream — at which point some tools may already have been called. Reporting error directly would cause state inconsistency.

7.7.3 Three-layer Compression System: Each with Its Own Role

Claude Code implements **four levels of context compression**, each targeting a different granularity:

```
HISTORY_SNIP        ── Coarsest: hard truncation by message count/token count
  Trigger: message count exceeds threshold
  Timing: before each Turn starts
```

```
  Characteristic: heuristic-based, doesn't understand semantics

Microcompact        ── Fine-grained: cache "expensive tool results"
  Trigger: tool result exceeds specific size
  Timing: lazy boundary (before next user message)
  Characteristic: replaced with summary, preserves information structure

Context Collapse   ── Medium-grained: commit "completed reasoning"
  Trigger: CONTEXT_COLLAPSE feature enabled
  Timing: phased commit
  Characteristic: replace verbose reasoning with verified conclusions

AutoCompact        ── Finest-grained: intelligent compression
  Trigger: autoCompactTracking reaches threshold
  Timing: at end of each Turn
  Characteristic: dynamic adjustment based on token budget
```

Key insight: These four levels of compression are not mutually exclusive, but **work together**. Low-level compression (HISTORY_SNIP) handles emergencies; high-level compression (AutoCompact) handles fine optimization.

7.7.4 Token Budget Design: Economics of Continuity

```
export async function checkTokenBudget(
  messages: Message[],
  budgetTracker: BudgetTracker,
): Promise<'continue' | 'stop'> {
  // Budget sufficient: continue
  // Budget exhausted: stop
  // But there's still "work worth continuing": can apply for extension
}
```

Token Budget is not just "how many tokens remain," but an **economic system**:

- Each Turn consumes budget (API calls + tokens)
- Budget exhaustion doesn't immediately stop — also judges "is it worth continuing"
- This is an **optimistic continue strategy**: continue unless there's clear evidence to stop

The 8k → 64k Max Output Tokens Recovery is also a similar economic decision:

- Default 8k is the cost/capability balance point
- When the model needs more space, progressively escalate to 64k
- Give up after 3 recovery attempts — this is an **exponential backoff** economic model

---

8. MCP Integration (Model Context Protocol)

8.1 MCP Client Architecture (`services/mcp/`)

ServerConnection State Machine

```
// Connection state types
type ConnectedMCPServer = {
  client: Client
  name: string
  capabilities: ServerCapabilities
  serverInfo?: { name: string; version: string }
  instructions?: string
  config: ScopedMcpServerConfig
  cleanup: () => Promise<void>
}

type FailedMCPServer = { name, type: 'failed', config, error? }
type NeedsAuthMCPServer = { name, type: 'needs-auth', config }
type PendingMCPServer = { name, type: 'pending', config, reconnectAttempt?, maxReconnectAttempts? }
type DisabledMCPServer = { name, type: 'disabled', config }
```

Note

The essence of connection state is a "graceful degradation spectrum": Connected / Pending / NeedsAuth / Failed / Disabled — five states covering the complete lifecycle of a distributed connection from establishment to death. The key design is the two intermediate states `NeedsAuth` and `Pending` — they distinguish "connection being processed" from "connection completely failed": the former means auto-recovery is possible, the latter means human intervention is needed. This **state subdivision rather than exception throwing** design allows the MCP client to make reasonable degradation behavior in any state, rather than categorizing all problems as "connection failure."

Server Config Schema

```
// STDIO transport
const McpStdioServerConfigSchema = z.object({
  type: z.literal('stdio').optional(),
  command: z.string().min(1),
  args: z.array(z.string()).default([]),
  env: z.record(z.string(), z.string()).optional(),
})

// SSE transport (with OAuth support)
const McpSSEServerConfigSchema = z.object({
  type: z.literal('sse'),
  url: z.string(),
  headers: z.record(z.string(), z.string()).optional(),
  headersHelper: z.string().optional(),     // Runtime-generated headers
  oauth: {
    clientId: z.string().optional(),
    callbackPort: z.number().optional(),
    authServerMetadataUrl: z.string().url().optional(),
    xaa: XaaConfig.optional(),
  }.optional(),
})

// HTTP polling transport
const McpHTTPServerConfigSchema = z.object({
  type: z.literal('http'),
  url: z.string(),
  headers: z.record(z.string(), z.string()).optional(),
  headersHelper: z.string().optional(),
  oauth: {...}.optional(),
```

```
})
```

MCP Tool Wrapper (MCPTool)

```
export const MCPTool = buildTool({
  isMcp: true,
  isOpenWorld() { return false },  // Non-open world
  name: 'mcp',
  maxResultSizeChars: 100_000,
  inputSchema: z.object({}).passthrough(),  // Accept any JSON
  outputSchema: z.string(),
  async checkPermissions() {
    return { behavior: 'passthrough', message: 'MCPTool requires permission.' }
  },
  renderToolUseMessage,        // UI rendering
  userFacingName: () => 'mcp',
  renderToolUseProgressMessage,
  renderToolResultMessage,
  isResultTruncated(output) { return isOutputLineTruncated(output) },
  mapToolResultToToolResultBlockParam(content, toolUseID) {
    return { tool_use_id: toolUseID, type: 'tool_result', content }
  },
})
```

8.2 MCP OAuth Authentication Flow

```
// McpAuthTool — dynamically created when server returns needs-auth
export function createMcpAuthTool(serverName, config): Tool

// Authentication flow:
// 1. Detect config type (SSE/HTTP support OAuth, STDIO doesn't)
// 2. Start OAuth flow (skipBrowserOpen: true)
// 3. Return authUrl to user
// 4. After user completes auth, server redirects to callbackPort
// 5. Background task: wait for auth completion → reconnect server
// 6. Server reconnected → replace stub tool with real tool
// 7. Cleanup stub tool (prefix mcp__<server>__authenticate)

type McpAuthOutput =
  | { status: 'auth_url', authUrl: string, message: string }
  | { status: 'unsupported', message: string }
  | { status: 'error', message: string }
```

8.3 MCP Resource Tools

```
// ListMcpResourcesTool — list available resources on MCP server
// ReadMcpResourceTool — read specific resource content
// Schema:
// ListMcpResourcesTool.input: { server?: string }
// ReadMcpResourceTool.input: { server: string, uri: string }
```

---

9. Skill System

9.1 Skill Definition

```
type BundledSkillDefinition = {
  name: string
  description: string
  aliases?: string[]
  whenToUse?: string         // Shown in Skill list
  argumentHint?: string       // Argument hint
  allowedTools?: string[]    // Restricted tools
  model?: string             // Model override
  disableModelInvocation?: boolean  // Only run hooks, don't call LLM
  userInvocable?: boolean    // Whether user-invocable (/skill-name)
  isEnabled?: () => boolean  // Feature gate
  hooks?: HooksSettings       // Session hooks
  context?: 'inline' | 'fork'  // Execution context
  agent?: string              // Dedicated Agent type
  files?: Record<string, string>  // Reference files
  getPromptForCommand: (args: string, context: ToolUseContext) => Promise<ContentBlockParam[]>
}
```

9.2 Skill Loading

Skill directory structure:

```
~/.claude/skills/                    ← User-level Skills
  my-skill/SKILL.md

<cwd>/.claude/skills/                ← Project-level Skills
  project-skill/SKILL.md

<policy>/.claude/skills/             ← Policy-level Skills
```

SKILL.md frontmatter fields:

```
---
displayName: My Skill                    # Optional: display name
description: What this Skill does            # Required
allowedTools: [Bash, Read, Write]      # Optional: tool restrictions
argumentHint: <args>                    # Optional: argument format
argumentNames: [name]                   # Optional: named parameters
whenToUse: When to use this               # Optional: usage hint
version: 1.0                           # Optional: version
model: sonnet-4                         # Optional: model
disableModelInvocation: false           # Optional: don't call LLM
```

```
userInvocable: true                     # Optional: user-invocable
hooks:                                  # Optional: Hooks
  PreToolUse:
    - hook: my-pre-hook
executionContext: fork                  # Optional: fork execution
agent: Explore                          # Optional: dedicated Agent
shell: bash | powershell                # Optional: shell type
paths:                                  # Optional: dynamic activation conditions
  - "**/*.ts"
  - "**/*.tsx"
---
# Skill body (optional)

This is the Skill description text.
```

9.3 SkillTool

```
const SkillTool = buildTool({
  name: 'Skill',
  aliases: ['skill'],

  inputSchema: z.object({
    name: z.string(),                   // Skill name
    args: z.string().optional(),        // Arguments
  }),

  async call({ name, args }, toolUseContext) {
    // 1. Load Skill definition
    const skill = await resolveSkill(name)

    // 2. Check userInvocable
    if (!skill.userInvocable) {
      return { success: false, error: '...' }
    }

    // 3. Register hooks (if defined)
    registerSessionHooks(rootSetAppState, agentId, skill.hooks)

    // 4. Get prompt
    const contentBlocks = await skill.getPromptForCommand(args ?? '', toolUseContext)

    // 5. Return result
    return { content: contentBlocks, skillName: name }
  },
})
```

Note

Skill is essentially an "LLM-level composition pattern": The fundamental difference between Skill and tool is — tools are actions Claude Code executes, Skills are **context instructions injected into LLM prompts**. SkillTool doesn't return execution results, but `contentBlocks` (a set of ContentBlockParam), which are appended directly to message history. This means the effect of a Skill depends entirely on whether the LLM "follows" it — no enforcement, only guidance. This is intentional: **Skill is a suggestion, not a command**. This design allows Skills to implement far more complex behavior than tools (e.g., the `simplify` skill can do complete code review), while maintaining the tool system's simplicity.

9.4 Built-in Bundled Skills

Skill	File	Description
`batch`	`bundled/batch.ts`	Batch operations
`claudeInChrome`	`bundled/claudeInChrome.ts`	Chrome integration
`debug`	`bundled/debug.ts`	Debug assistant
`update-config`	`bundled/updateConfig.ts`	Config update
`loop`	`bundled/loop.ts`	Loop tasks
`simplify`	`bundled/simplify.ts`	Code simplification
`remember`	`bundled/remember.ts`	Memory management
`scheduleRemoteAgents`	`bundled/scheduleRemoteAgents.ts`	Remote agent scheduling

9.5 Dynamic Skill Discovery

When the user edits files matching `paths` conditions, Skills can auto-activate (dynamic discovery mechanism). Uses gitignore-style path matching.

---

10. Plugin System

10.1 Plugin Manifest Schema

```
const PluginManifestSchema = z.object({
  name: z.string().min(1),
  version: z.string().optional(),
  description: z.string().optional(),
  author: PluginAuthorSchema().optional(),
  homepage: z.string().url().optional(),
  repository: z.string().optional(),
  license: z.string().optional(),
  keywords: z.array(z.string()).optional(),
  dependencies: z.array(DependencyRefSchema()).optional(),

  // === Optional components ===
  hooks: PluginManifestHooksSchema().partial(),
  commands: PluginManifestCommandsSchema().partial(),
  agents: PluginManifestAgentsSchema().partial(),
  skills: PluginManifestSkillsSchema().partial(),
  outputStyles: PluginManifestOutputStylesSchema().partial(),
  channels: PluginManifestChannelsSchema().partial(),
  mcpServers: PluginManifestMcpServerSchema().partial(),
  lspServers: PluginManifestLspServerSchema().partial(),
  settings: PluginManifestSettingsSchema().partial(),
  userConfig: PluginManifestUserConfigSchema().partial(),
})
```

10.2 Loaded Plugin Types

```
type LoadedPlugin = {
  name: string
  manifest: PluginManifest
  path: string
  source: string           // e.g., "name@marketplace"
  repository: string

  // Optional components
  enabled?: boolean
  isBuiltin?: boolean
  sha?: string
  commandsPath?: string
  agentsPath?: string
  skillsPath?: string
  outputStylesPath?: string
  hooksConfig?: HooksSettings
  mcpServers?: Record<string, McpServerConfig>
  lspServers?: Record<string, LspServerConfig>
  settings?: Record<string, unknown>
}
```

10.3 Plugin Source Types

```
// Builtin plugin (code-embedded)
type BuiltinPluginDefinition = {
  name: string
  description: string
  version?: string
  skills?: BundledSkillDefinition[]
  hooks?: HooksSettings
  mcpServers?: Record<string, McpServerConfig>
  isAvailable?: () => boolean
  defaultEnabled?: boolean
}

// Remote sources
const PluginSourceSchema = z.union([
  RelativePath(),                     // Local relative path
  { source: 'npm', package: string, version?: string, registry?: string },
  { source: 'pip', package: string, version?: string, registry?: string },
  { source: 'url', url: string, ref?: string, sha?: string },
  { source: 'github', repo: string, ref?: string, sha?: string },
  { source: 'git-subdir', url: string, path: string, ref?: string, sha?: string },
])

// Marketplace sources
const MarketplaceSourceSchema = z.discriminatedUnion('source', [
  { source: 'url', url: string, headers?: Record<string, string> },
  { source: 'github', repo: string, ref?: string, path?: string, sparsePaths?: string[] },
  { source: 'git', url: string, ref?: string, path?: string, sparsePaths?: string[] },
  { source: 'npm', package: NpmPackageNameSchema() },
  { source: 'file', path: string },
  { source: 'directory', path: string },
  { source: 'hostPattern', hostPattern: string },
  { source: 'pathPattern', pathPattern: string },
  { source: 'settings', name: string, plugins: array, owner?: PluginAuthor },
])
```

10.4 Plugin Loading Pipeline

```
export async function loadAllPlugins(): Promise<PluginLoadResult> {
  // 1. Load Marketplace plugins (with dependency resolution)
  // 2. Load session-only plugins (--plugin-dir)
  // 3. Load builtin plugins
  // 4. Merge deduplicate (by name)
  // 5. Validate dependencies
  // 6. Cache plugin settings
}

type PluginLoadResult = {
  enabled: LoadedPlugin[]
  disabled: LoadedPlugin[]
  errors: PluginError[]
}
```

10.5 Plugin Error Types

```
type PluginError =
  | { type: 'path-not-found', source, plugin, path, component }
  | { type: 'git-auth-failed', source, plugin, gitUrl, authType }
  | { type: 'git-timeout', source, plugin, gitUrl, operation }
  | { type: 'network-error', source, plugin, url, details? }
  | { type: 'manifest-parse-error', source, plugin, manifestPath, parseError }
  | { type: 'manifest-validation-error', source, plugin, manifestPath, validationErrors }
  | { type: 'plugin-not-found', source, pluginId, marketplace }
  | { type: 'marketplace-not-found', source, marketplace, availableMarketplaces }
  | { type: 'marketplace-load-failed', source, marketplace, reason }
  | { type: 'mcp-config-invalid', source, plugin, serverName, validationError }
  | { type: 'hook-load-failed', source, plugin, hookPath, reason }
  | { type: 'component-load-failed', source, plugin, component, path, reason }
  | { type: 'marketplace-blocked-by-policy', source, plugin?, marketplace, blockedByBlocklist?, allowedSources }
  | { type: 'dependency-unsatisfied', source, plugin, dependency, reason }
  | { type: 'plugin-cache-miss', source, plugin, installPath }
  | { type: 'generic-error', source, plugin?, error }
```

---

11. Permission and Security Model

11.1 Permission Modes

```
// Externally visible modes (user-configurable)
const EXTERNAL_PERMISSION_MODES = [
  'acceptEdits',    // Auto-accept all edits
  'bypassPermissions', // Skip all permission checks
  'default',        // Standard mode
  'dontAsk',        // Don't ask (silent deny)
  'plan',           // Plan mode
] as const

// Internal modes (with auto-classification)
const INTERNAL_PERMISSION_MODES = [
  ...EXTERNAL_PERMISSION_MODES,
  ...(feature('TRANSCRIPT_CLASSIFIER') ? (['auto'] as const) : ([] as const)),
  'bubble',  // Bubble mode
] as const

type PermissionMode = InternalPermissionMode
```

11.2 Permission Behaviors

```
type PermissionBehavior = 'allow' | 'deny' | 'ask'
```

11.3 Permission Decision Types

```
// Allow
type PermissionAllowDecision<Input> = {
  behavior: 'allow'
  updatedInput?: Input
  userModified?: boolean
  decisionReason?: PermissionDecisionReason
  toolUseID?: string
  acceptFeedback?: string
  contentBlocks?: ContentBlockParam[]
}

// Ask
type PermissionAskDecision<Input> = {
  behavior: 'ask'
  message: string
  updatedInput?: Input
  decisionReason?: PermissionDecisionReason
  suggestions?: PermissionUpdate[]       // Suggested updates
  blockedPath?: string                    // Blocked path
  metadata?: PermissionMetadata
  isBashSecurityCheckForMisparsing?: boolean
  pendingClassifierCheck?: PendingClassifierCheck
  contentBlocks?: ContentBlockParam[]
}

// Deny
type PermissionDenyDecision = {
  behavior: 'deny'
  message: string
  decisionReason: PermissionDecisionReason
  toolUseID?: string
}

// Passthrough (from Hook)
type PermissionPassthroughDecision = {
  behavior: 'passthrough'
  message: string
  decisionReason?: PermissionDecisionReason
  suggestions?: PermissionUpdate[]
  blockedPath?: string
  pendingClassifierCheck?: PendingClassifierCheck
}
```

11.4 Permission Decision Reasons

```
type PermissionDecisionReason =
  | { type: 'rule', rule: PermissionRule }              // Rule matched
  | { type: 'mode', mode: PermissionMode }               // Permission mode
  | { type: 'subcommandResults', reasons: Map }           // Subcommand results
  | { type: 'permissionPromptTool', toolName, toolResult } // Permission prompt tool
  | { type: 'hook', hookName, hookSource?, reason? }     // Hook interception
  | { type: 'asyncAgent', reason: string }               // Async agent
  | { type: 'sandboxOverride', reason }                  // Sandbox override
  | { type: 'classifier', classifier: string, reason }    // Auto-classifier
  | { type: 'workingDir', reason: string }               // Working directory
  | { type: 'safetyCheck', reason, classifierApprovable }  // Safety check
  | { type: 'other', reason: string }                    // Other
```

11.5 Auto Classifier

```
type YoloClassifierResult = {
  thinking?: string
  shouldBlock: boolean
  reason: string
  unavailable?: boolean          // Classifier unavailable
  transcriptTooLong?: boolean    // Transcript exceeds limit
  model: string                  // Model used
  usage?: ClassifierUsage
  durationMs?: number
  promptLengths?: {
    systemPrompt: number
    toolCalls: number
    userPrompts: number
  }
  errorDumpPath?: string
  stage?: 'fast' | 'thinking'    // Two-stage classification
  stage1Usage?: ClassifierUsage; stage1DurationMs?: number
```

```
  stage2Usage?: ClassifierUsage; stage2DurationMs?: number
}

type ClassifierUsage = {
  inputTokens: number
  outputTokens: number
  cacheReadInputTokens: number
  cacheCreationInputTokens: number
}
```

Note

Two-stage classification is "speed vs. accuracy" expressed at the protocol level: the `fast` stage returns a decision before the permission dialog appears (based on heuristics/lightweight model), while the `thinking` stage deploys full reasoning capability for deep analysis. The two stages are not "first fast then slow" but "first guess then verify" — `fast` stage decisions take effect immediately (user experiences instant response), `thinking` stage conclusions update subsequent decision models or log audit entries. Notably, when `stage2`'s `thinking` classification result differs from `stage1`, the system doesn't retract the already-displayed UI — this means **errors are visible, but responses are timely**, which is another pragmatic trade-off between user experience and security.

11.6 Permission Context

```
type ToolPermissionContext = {
  readonly mode: PermissionMode
  readonly additionalWorkingDirectories: ReadonlyMap<string, AdditionalWorkingDirectory>
  readonly alwaysAllowRules: ToolPermissionRulesBySource  // Allowlist rules
  readonly alwaysDenyRules: ToolPermissionRulesBySource   // Denylist rules
  readonly alwaysAskRules: ToolPermissionRulesBySource     // Force-ask rules
  readonly isBypassPermissionsModeAvailable: boolean
  readonly strippedDangerousRules?: ToolPermissionRulesBySource
  readonly shouldAvoidPermissionPrompts?: boolean
  readonly awaitAutomatedChecksBeforeDialog?: boolean
  readonly prePlanMode?: PermissionMode
}
```

11.7 Hook Permission Decision Resolution

Key invariant: Hook's 'allow' does not bypass deny/ask rules in `settings.json`.

```
resolveHookPermissionDecision(hookResult, tool, input, context, canUseTool, ...) {
  if (hookResult?.behavior === 'allow') {
    // Hook says allow, but rules still need checking
    if (tool.requireCanUseTool || context.requireCanUseTool) {
      return canUseTool(input)  // Force call canUseTool
    }
    // Rule checking (deny/ask rules still apply!)
    const denyRule = getDenyRuleForTool(context, tool)
    if (denyRule) return denyRule
    const askRule = getAskRuleForTool(context, tool)
    if (askRule) return askRule
    return hookResult  // Allow
  }
  // Hook says deny/ask or no hook: go through normal permission flow
  return normalPermissionFlow(...)
}
```

11.8 Bash Tool Security Analysis

BashTool uses multiple mechanisms to ensure command safety:

```
// Command classification (isSearchOrReadBashCommand)
const BASH_SEARCH_COMMANDS  = new Set(['find', 'grep', 'rg', 'ag', 'ack', ...])
const BASH_READ_COMMANDS    = new Set(['cat', 'head', 'tail', 'less', 'more', 'wc', ...])
const BASH_LIST_COMMANDS    = new Set(['ls', 'tree', 'du'])
const BASH_SILENT_COMMANDS  = new Set(['mv', 'cp', 'rm', 'mkdir', 'chmod', ...])

// AST security parsing (tree-sitter)
// 1. parseForSecurity(command) — parse command AST
// 2. Extract all subcommands and their arguments
// 3. Validate path constraints (deny rules)
// 4. Validate sed constraints

// Safe environment variables
const SAFE_ENV_VARS = new Set([
  'GOOS', 'GOARCH', 'NODE_ENV', 'PYTHONUNBUFFERED',
  'RUST_BACKTRACE', 'RUST_LOG', 'LANG', 'LC_ALL',
  'TERM', 'TZ', 'LS_COLORS', 'LSCOLORS', 'GREP_COLORS',
])

// Auto-background trigger conditions
// 1. Command has timeout exceeding limit
// 2. Exceeds ASSISTANT_BLOCKING_BUDGET_MS (15s)
// 3. Explicit run_in_background: true
```

---

12. Core Type Definitions

12.1 Tool-related Core Types

Type	Definition Location	Description
`Tool`	`Tool.ts`	Tool main interface (40+ properties)
`ToolResult<T>`	`Tool.ts`	Tool execution result (with new messages, context modifiers)
`ToolProgressData`	`Tool.ts`	Tool progress data type
`ToolUseContext`	`Tool.ts`	Tool execution context (includes AppState, AbortController, messages, etc.)
`ToolInputJSONSchema`	`Tool.ts`	JSON Schema representation
`ValidationResult`	`Tool.ts`	Input validation result
`PermissionResult`	`types/permissions.ts`	Permission check result
`ToolPermissionContext`	`types/permissions.ts`	Permission config context
`YoloClassifierResult`	`types/permissions.ts`	Auto-classifier result

12.2 Agent-related Core Types

Type	Definition Location	Description
`AgentDefinition`	`loadAgentsDir.ts`	Agent definition interface (supports builtin/custom/plugin)
`BuiltInAgentDefinition`	`loadAgentsDir.ts`	Built-in Agent definition (supports getSystemPrompt callback)
`AgentSpec`	`agentToolUtils.ts`	Agent specification
`AgentContext`	`utils/agentContext.ts`	Agent execution context (subagent / teammate)
`SubagentContext`	`utils/agentContext.ts`	Subagent context
`TeammateAgentContext`	`utils/agentContext.ts`	Teammate Agent context
`AgentMemoryScope`	`agentMemory.ts`	Memory scope ('user'/'project'/'local')
`SpawnTeammateConfig`	`spawnMultiAgent.ts`	Teammate spawn config
`AgentToolResult`	`AgentTool.tsx`	Agent tool result (completed/async_launched/teammate_spawned/remote_launched)

12.3 Task-related Core Types

Type	Description
`LocalAgentTaskState`	Complete state of local background Agent task
`AgentProgress`	Real-time progress (toolUseCount/tokenCount/activities)
`ProgressTracker`	Progress tracker
`AgentActivity`	Tool activity record

12.4 SDK Types (`entrypoints/agentSdkTypes.ts`)

Claude Code provides complete SDK type exports:

- **Core types**: messages, tool results, Agent configuration
- **Runtime types**: Session management, error handling
- **Settings types**: config schema validation
- **Permission types**: PermissionMode, PermissionResult

---

13. Design Philosophy Summary

13.1 Tool Design Philosophy

Philosophy	Practice
Schema-first	Each tool has Zod input/output schema; LLM auto-gets type awareness
Self-describing	`description()` and `prompt()` methods let tools describe themselves; no extra prompts needed
Multi-dimensional safety	Multi-layer protection via `isReadOnly`/`isDestructive`/`checkPermissions`
Concurrency-aware	`isConcurrencySafe` allows parallel execution of independent operations, significantly improving throughput
Deferred loading	`shouldDefer` mechanism avoids loading infrequently used heavy tools
UI rendering separation	Each tool handles its own React rendering logic; CLI and UI share the same tool set
Fail-closed default	Defaults are conservative (not concurrent, has write operations); each tool overrides as needed

13.2 Agent Design Philosophy

Philosophy	Practice
Metadata-driven	Agent definitions are data (YAML/Markdown), not hardcoded; highly extensible
Role specialization	Specialized agents like Explore/Plan/Verification replace general-purpose agents
Context inheritance	Fork subagent auto-inherits parent agent context; no redundant passing
Resource isolation	Different agent types have different toolset constraints (`ALL_AGENT_DISALLOWED_TOOLS`, etc.)
Memory persistence	Agent memory supports cross-session context (three levels: user/project/local)
Snapshot system	`agentMemorySnapshot` periodically backs up; supports recovery and sync
One-shot optimization	High-frequency agents like Explore/Plan skip trailer; saves significant tokens

13.3 Multi-Agent Collaboration Philosophy

Philosophy	Practice
Unified communication protocol	SendMessageTool + `<task-notification>` XML is the standard for inter-Agent communication
Backend abstraction	Tmux/iTerm2/InProcess backends via unified interface; support different use cases
Orchestration layer separation	Coordinator mode separates "thinking/planning" and "execution"
Progressive authorization	Tools can configure allow/deny rules; achieve fine-grained permission delegation
Async execution	Teammate tasks are fully async; communicate via mailbox mechanism
Graceful shutdown protocol	shutdown_request/response bidirectional handshake ensures clean exit

13.4 Query Engine Design

Philosophy	Practice
AsyncGenerator everywhere	All message streams are AsyncGenerator; easy streaming processing and cancellation
Multi-stage recovery	PTL Recovery → Reactive Compact → Escalate three-layer degradation
Auto-compression	HISTORY_SNIP / Microcompact / Context Collapse / AutoCompact multi-level compression
Token Budget	Budget-based continue/terminate decision; optimizes token consumption
Streaming First	Tools execute concurrently; results buffered in enqueue order then yielded

13.5 MCP Ecosystem Design

Philosophy	Practice
Bidirectional role	Both MCP client (using other servers) and MCP server (供外部调用)
Dynamic tool creation	Functions like `createMcpAuthTool` dynamically create tools
Native OAuth	SSE/HTTP transport supports complete OAuth authentication flow
Multi-transport support	STDIO / SSE / HTTP / WebSocket / SDK multiple transport methods

13.6 Plugin Architecture

Philosophy	Practice
Complete manifest schema	Plugins can declare: commands/agents/skills/hooks/mcpServers/lspServers
Multi-source loading	npm / git / url / github / local directory / Marketplace
Dependency resolution	Auto-resolve and load plugin dependencies
Policy integration	Deep integration with policy settings

13.7 Hook System

```
// PreToolUse Hooks
//   - message: append message
//   - hookPermissionResult: provide permission decision
//   - hookUpdatedInput: modify tool input
//   - preventContinuation: prevent continuation
//   - stopReason: record stop reason
//   - additionalContext: append context
//   - stop: stop execution immediately

// PostToolUse Hooks
//   - message: append message
//   - blockingError: prevent continuation
//   - preventContinuation: prevent continuation
//   - updatedMCPToolOutput: update MCP tool output

// Lifecycle Hooks
//   SubagentStart → Agent runs → SubagentStop
//   Stop → tool call → Stop → Session closes
```

13.8 Global Design Philosophy of the Tool System

Claude Code's tool system design follows a core principle: **make tools as transparent as possible, while retaining sufficient control**.

Transparency体现在:

1. **Zod Schema self-description**: Tool's input format is both type constraint and LLM prompt source
2. **`description()` method**: Each tool knows how to describe itself
3. **`renderToolUseMessage` and other rendering methods**: Tool's UI representation is handled by the tool itself

Control体现在:

1. **Metadata properties like `isConcurrencySafe`**: Execution engine controls concurrency via metadata, not tool implementation
2. **Tool pool assembly order**: Deterministic composition via `uniqBy` and `sort`
3. **Alias rejection**: Built-in tools always take priority over same-name MCP tools

Deep implication of this principle: Tool developers only need to care about "what the tool does," while "how the tool is used" is decided by the framework. This is a typical application of the **Hollywood Principle** (Don't call us, we'll call you).

13.9 Global Design Philosophy of the Agent System

Claude Code's Agent system can be understood as a **layered authorization system**:

```
User
  │
  ├── Authorize Claude Code → default capabilities
  │
  ├── Authorize AgentTool → launch sub-agents (with different tool sets)
  │
  ├── Authorize TeamCreateTool → create teams (multi-Agent collaboration)
  │
  └── Authorize Coordinator mode → delegate orchestration to sub-agent
```

Each authorization layer comes with constraints:

- Subagent has `ALL_AGENT_DISALLOWED_TOOLS`
- Team members have `IN_PROCESS_TEAMMATE_ALLOWED_TOOLS`
- Coordinator has `COORDINATOR_MODE_ALLOWED_TOOLS`

The beauty of this design: Authorization is composable. User's authorization to Claude Code, after multi-layer distribution, ultimately reaches the **minimum necessary permission subset**.

Fork's Context Inheritance Model

Fork subagent's context inheritance is not a simple "copy-paste," but a **controlled view sharing**:

```
// Fork inherits parent agent's message history (for understanding context)
// But fork has its own AbortController (for independent control)
// Fork notifies parent agent on completion (instead of returning directly)
```

This is similar to React's "state lifting" pattern: shared data is maintained at the parent level; children get references, not copies.

13.10 Architectural Insights on Multi-Agent Collaboration

Claude Code's multi-Agent collaboration system (Swarm) solves three fundamental problems:

1. Communication problem: How do agents on different processes, different machines communicate?

- **Solution**: Mailbox file + polling (tmux/iTerm2) or API relay (remote)
- **Design insight**: Communication medium is pluggable (`PaneBackend` interface)

2. Isolation problem: How to run multiple agents in a shared Node.js process without interfering with each other?

- **Solution**: `AsyncLocalStorage` for context isolation
- **Design insight**: This is a model of "controlled sharing" — process resources (API client, MCP connections) are shared, but logical context is isolated

3. Lifecycle problem: How to gracefully start and terminate agents?

- **Solution**: Structured Shutdown protocol (request/response handshake)
- **Design insight**: Two-sided confirmation ensures both parties know the session has ended

13.12 Task Framework: State Machine as Lifecycle

LocalAgentTask's state machine design reveals an important pattern: **using TypeScript types to constrain state transitions**.

```
type LocalAgentTaskState = TaskStateBase & {
  type: 'local_agent'
  status: 'running' | 'pending' | 'completed' | 'failed' | 'killed'
  isBackgrounded: boolean  // Running state × background state combination
  pendingMessages: string[]  // Undelivered message queue
  evictAfter?: number       // GC timestamp
  // ...
```

```
}
```

The elegance of state machine design:

`isBackgrounded` is not an independent state, but a **dimensional marker of state** — this allows the same "running" state to have two behaviors:

- Foreground running: real-time streaming output to CLI
- Background running: silent execution; results written to disk

GC strategy balanced with user experience:

```
setTaskState(taskId, setAppState, task => ({
  ...task,
  status: 'killed',
  evictAfter: Date.now() + PANEL_GRACE_MS,  // Keep for 5 minutes
}))
```

After a task is killed, the UI still displays it for 5 minutes — this is "graceful degradation": users may need to review killed task results for follow-up decisions.

13.13 Permission System: Layered Decision Tree

Claude Code's permission decision is a **multi-source decision tree**, priority from high to low:

```
Tool call request
  │
  ├─ [1] Hook interception?
  │      └─ Hook says allow → go to rule checking
  │      └─ Hook says deny/ask → adopt directly
  │
  ├─ [2] Rule checking (alwaysDeny / alwaysAsk / alwaysAllow)
  │      └─ deny/ask → adopt directly
  │      └─ allow → go to mode checking
  │
  ├─ [3] Permission mode checking (acceptEdits / bypassPermissions / dontAsk)
  │      └─ bypassPermissions → allow directly
  │      └─ acceptEdits → allow edit operations
  │      └─ dontAsk → deny directly
  │      └─ default → go to classifier
  │
  └─ [4] Classifier decision (auto mode)
         └─ yolo (auto-allow) → allow
         └─ safe → ask
         └─ blocked → deny
```

Key invariant: Rules (step 2) can never be bypassed by Hooks (step 1). This means even if you install a Hook that "allows all operations," the user's `deny` rules in `settings.json` still take effect.

13.14 Memory System: Three Paradigms for Persistent Context

Claude Code's Agent memory system supports three persistence paradigms, corresponding to different sharing scopes:

Scope	Storage Location	Sharing Granularity	VCS-friendly	Use Case
`user`	`~/.claude/`	Cross-project, cross-machine	❌	User preferences, coding style
`project`	`<cwd>/.claude/`	Shared within project	✅	Project context, code standards
`local`	`<cwd>/.claude/`	Machine-specific	❌	Machine-specific information

Design insight: Both `project` and `local` are stored under `.claude/`, but distinguished by different subdirectories — this allows `.claude/` to be in VCS, while `.claude/agent-memory-local/` is in `.gitignore`.

Snapshot system (`agentMemorySnapshot`) is the safety net of the memory system: periodically writes current memory to JSON snapshot, preventing long-running agents from losing context after crashes.

13.11 Fault Tolerance Philosophy of the Query Engine

Claude Code's query engine follows the principle of **"don't abandon, don't give up"** when handling errors:

Don't abandon:

- PTL errors are withheld, not immediately reported
- Tombstone messages used as placeholders during streaming fallback
- Orphaned messages are retained (tombstone) rather than deleted

Don't give up:

- Three recovery chances: 8k → 64k → multi-turn
- Three-layer degradation for prompt-too-long (collapse → compact → escalate)
- Streaming fallback for model switching

Root of this fault tolerance philosophy: "Errors" in LLM applications are often temporary (token limit, resource shortage) rather than permanent (type error, logic error). Therefore, try recovery first; only expose errors when truly necessary.

13.8 Engineering Highlights

Aspect	Practice
Type safety	Full TypeScript + Zod Schema; complete input/output coverage
Feature gating	`isEnabled()` method enables tools/features dynamically by environment/config
Testability	`TestingPermissionTool` and other test tools support permission system validation
Prompt Cache	Fork shares cache benefit; auto eviction hint
Analytics	Every key path records analytics events (tengu_* series)
Performance tracking	Perfetto tracing, TTFT metrics, Token counting, tool duration
Session persistence	Sidechain transcript + SessionStorage support session recovery

---

Appendix: Key Files Index

Tool Core

File	Description
`src/Tool.ts`	Tool interface definition, `buildTool` factory, ToolUseContext
`src/tools.ts`	Tool pool assembly, `getAllBaseTools`, `assembleToolPool`
`src/constants/tools.ts`	All tool constants, access control sets

Tool Execution

File	Description
`src/services/tools/toolExecution.ts`	`runToolUse`, `checkPermissionsAndCallTool` complete implementation
`src/services/tools/StreamingToolExecutor.ts`	Concurrent executor, Queue management
`src/services/tools/toolHooks.ts`	PreToolUse/PostToolUse Hooks execution
`src/tools/BashTool/BashTool.tsx`	Bash tool complete implementation
`src/tools/FileEditTool/FileEditTool.ts`	File edit tool complete implementation

Agent Core

File	Description
`src/tools/AgentTool/AgentTool.tsx`	Agent tool main implementation (125K tokens)
`src/tools/AgentTool/runAgent.ts`	Agent running logic, context initialization
`src/tools/AgentTool/agentToolUtils.ts`	Progress tracking, lifecycle hooks, result finalization
`src/tools/AgentTool/loadAgentsDir.ts`	Agent definition loading, Zod schema validation
`src/tools/AgentTool/forkSubagent.ts`	Fork subagent
`src/tools/AgentTool/agentMemory.ts`	Agent memory system
`src/tools/AgentTool/agentMemorySnapshot.ts`	Agent memory snapshot
`src/tools/AgentTool/prompt.ts`	Agent tool prompt generation
`src/tools/AgentTool/constants.ts`	Agent tool constants
`src/tools/AgentTool/built-in/*.ts`	Built-in agent definitions

Multi-Agent

File	Description
`src/tools/shared/spawnMultiAgent.ts`	Teammate spawn core logic
`src/tools/TeamCreateTool/`	Team creation
`src/tools/SendMessageTool/`	Message sending (including structured protocol)
`src/utils/swarm/`	TmuxBackend / ITermBackend / InProcessBackend
`src/utils/agentContext.ts`	AsyncLocalStorage context isolation

Task Framework

File	Description
`src/tasks/LocalAgentTask/LocalAgentTask.tsx`	Local task state machine
`src/tasks/LocalAgentTask/framework.ts`	Generic task registration/tracking
`src/tasks/RemoteAgentTask/`	Remote tasks
`src/tasks/InProcessTeammateTask/`	In-process teammate tasks

Query Engine

File	Description
`src/QueryEngine.ts`	QueryEngine main class (46K tokens)
`src/query.ts`	Main query loop (68K tokens)
`src/coordinator/coordinatorMode.ts`	Coordinator mode

MCP

File	Description
`src/services/mcp/client.ts`	MCP client connection management
`src/services/mcp/types.ts`	MCP protocol types
`src/tools/MCPTool/`	MCP tool wrapper
`src/tools/McpAuthTool/`	MCP OAuth authentication

Skill / Plugin

File	Description
`src/skills/`	Skill system
`src/plugins/`	Plugin system

SDK

File	Description
`src/entrypoints/agentSdkTypes.ts`	SDK type exports
`src/entrypoints/mcp.ts`	MCP server entry

---

14. In-Depth Topic Analysis

> The following content provides supplementary in-depth analysis of the topics covered in previous chapters, including code-level design philosophy analysis, deep motivations behind architectural decisions, and engineering culture embodied in the source code.

14.1 Six Design Principles of the Tool System

Through source code analysis, we distill six design principles Claude Code's tool system follows:

Principle One: Schema-first, Types are Documentation

Each tool's Zod schema is both a runtime type validator and the LLM's tool description source. This eliminates the "documentation and code inconsistency" problem — **no extra documentation; the schema is the documentation**.

Principle Two: Metadata-driven Behavior Control

Tool behavior (whether concurrent, read-only, destructive) is controlled via metadata, not hardcoding. This design enables:

- Execution engine can uniformly handle all tools
- Tool behavior can be dynamically adjusted at runtime
- Security policies can be centrally managed

Principle Three: Fail-closed Defaults

```
isConcurrencySafe: () => false,   // Not concurrent by default
isReadOnly:      () => false,   // Has write operations by default
isDestructive:   () => false,   // Non-destructive by default
```

These defaults may seem conservative, but they embody the **principle of least surprise**: tools should default to safest behavior, only change when clearly able to be more aggressive.

Principle Four: Speculative Execution for Latency Reduction

Bash classifier check starts in parallel with permission check — this leverages the assumption "user won't cancel." If the user waits for classifier result, the permission check is already running.

Principle Five: Tombstone, Not Discard

In streaming processing, processed messages are marked as "tombstone" (keeping placeholder) rather than deleted. This ensures message index integrity and avoids index misalignment caused by deletion.

Principle Six: Determinism Over Performance

Built-in tools and MCP tools sorted by `name.sort()` and then concatenated — these constraints ensure sessions with the same configuration produce the same prompt, fundamentally guaranteeing Prompt Cache hit rate.

14.2 Six Design Principles of the Agent System

Principle One: Metadata is Configuration

Agent definitions are YAML/Markdown files, not TypeScript code. This means:

- Non-engineers can define agents
- Agent definitions can be dynamically updated (no recompilation needed)
- Agent definitions can be version-controlled

Principle Two: Context Inheritance Over Copying

Fork subagent inherits parent agent context — this avoids explicitly passing messages every time a subagent is launched. Context is passed by "reference" rather than "value," saving copy overhead.

Principle Three: Resource Isolation Used as Needed

Worktree isolation, In-Process isolation, tmux isolation — different isolation levels correspond to different resource costs. Claude Code's design allows users to choose the appropriate isolation level, rather than forcing the heaviest option.

Principle Four: Composable Lifecycle Hooks

Pre-spawn Hooks, Session Hooks, Post-completion Hooks — these hooks cover the complete lifecycle of an agent, allowing external systems to intervene at any stage. This is an AOP-style design of "cross-cutting concerns."

Principle Five: Toolset Constraints Over Trust

Subagents don't get all parent agent tools by default. Toolset for each agent type is explicitly limited via sets like `ALL_AGENT_DISALLOWED_TOOLS`, `ASYNC_AGENT_ALLOWED_TOOLS` — this embodies the **principle of least privilege**.

Principle Six: Async by Default

All agents' default behavior is async (`run_in_background` is the default option); foreground running is just a special "waiting" mode. This ensures Claude Code can manage large numbers of agents without blocking the main thread.

14.3 "Magic Comments" in Source Code: Reflection of Engineering Culture

Claude Code's source code has unique comment styles, reflecting the engineering team's culture:

1. "Wizard" comments (query.ts):

```
/**
 * The rules of thinking are lengthy and fortuitous...
 * Heed these rules well, young wizard...
 */
```

These humorous comments appear around complex logic (thinking block processing rules), possibly hinting this code underwent multiple rounds of debugging and refactoring.

2. "This is a bit of a hack" comments (toolExecution.ts):

```
// This is a bit of a hack to get progress events and final results
// into a single async iterable.
```

The team doesn't hide the existence of "hacks," but explicitly labels them and explains the reason. This is **honest engineering**.

3. "Why not just" comments: Source code contains many comments explaining "why not take a simpler approach," helping later maintainers understand the context of design decisions.

4. Permanent TODO comments: Some TODOs have `[NEED WORK]` or `[TBD]` tags, indicating "we know this isn't perfect, but acceptable for the current version." This is a **pragmatic** engineering attitude.

14.4 Quick Reference: Design Intent of Key Files

File	Design Intent
`Tool.ts`	Complete contract of tool interface; the "type constitution" of the entire tool system
`StreamingToolExecutor.ts`	Concurrency control engine for tool execution; implementation of "minimum safety window" mode
`toolExecution.ts`	Complete lifecycle of tool invocation; includes permission checking and error handling
`toolHooks.ts`	Execution engine of hook system; supports Pre/Post Tool Use hooks
`tools.ts`	Tool pool assembly logic; key to Prompt Cache-friendly design
`query.ts`	State machine of main query loop; the brain of entire Claude Code
`QueryEngine.ts`	Externally exposed query interface; encapsulates query loop implementation details
`runAgent.ts`	Agent startup and running logic; includes context initialization and cleanup
`spawnMultiAgent.ts`	Teammate spawn flow; call site of Swarm backend abstraction
`agentContext.ts`	AsyncLocalStorage wrapper; standard answer for in-process isolation
`constants/tools.ts`	All tool constants; centralized declaration of security policies
`prompt.ts` (Agent)	Agent prompt generation; includes critical text like Fork guidance

14.5 Architecture Decision Records (ADR)

The following key architectural decisions are distilled from source code:

ADR-1: Tool pool sorted alphabetically

- **Decision**: Built-in tools and MCP tools sorted by `name.sort()` then concatenated

- **Reason**: Ensure sessions with the same tool set share Prompt Cache
- **Trade-off**: MCP tools' relative position is unstable (depends on MCP server count)
- **Status**: Adopted

ADR-2: AsyncGenerator as message stream format

- **Decision**: All internal message passing uses `AsyncGenerator<Message>` instead of Promise
- **Reason**: Support streaming consumption, interruption, and pipeline composition
- **Trade-off**: Call chain debugging is difficult (stack broken by async generator)
- **Status**: Adopted

ADR-3: Hook's allow does not bypass rules' deny

- **Decision**: `resolveHookPermissionDecision` still checks rules after Hook allow
- **Reason**: Security policy must be centrally controlled; cannot be accidentally overridden by Hook
- **Trade-off**: Hook cannot fully control permission decisions
- **Status**: Adopted

ADR-4: Fork skips SendMessage trailer

- **Decision**: Explore/Plan agents (one-shot) skip full communication protocol handshake
- **Reason**: These agents are known not to continue communication; saves ~135 tokens
- **Trade-off**: Cannot continue Explore/Plan agent via SendMessage
- **Status**: Adopted

ADR-5: Bash error cancels all concurrent tools

- **Decision**: When Bash tool errors, cancel all concurrent tools via `siblingAbortController`
- **Reason**: Bash command chains usually have implicit dependencies (mkdir fails → subsequent commands meaningless)
- **Trade-off**: Independent Bash commands (e.g., parallel curl) may also be mistakenly killed
- **Status**: Adopted; Read/WebFetch unaffected